

A Survey of RISC-V Secure Enclaves and Trusted Execution Environments

Marouene Boubakri, Belhassen Zouari

▶ **To cite this version:**

Marouene Boubakri, Belhassen Zouari. A Survey of RISC-V Secure Enclaves and Trusted Execution Environments. Electronics, 2025, 14 (21:4171), ⟨10.3390/electronics14214171⟩. ⟨hal-05335088⟩

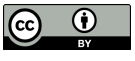
HAL Id: hal-05335088

https://hal.science/hal-05335088v1

Submitted on 28 Oct 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Distributed under a Creative Commons CC BY 4.0 - Attribution - International License

electronics

MDPI

Article

A Survey of RISC-V Secure Enclaves and Trusted Execution Environments

Marouene Boubakri [1,*] and Belhassen Zouari [2]

[1] Mediatron Lab, SupCom, University of Carthage, Ariana 2083, Tunisia
[2] ICL, Junia, Catholic University of Lille, LITL, F-59000 Lille, France
* Correspondence: marouene.boubakri@ept.u-carthage.tn; Tel.: +216-99-283-600

Abstract

RISC-V has emerged as a compelling alternative to proprietary instruction set architectures, distinguished by its openness, extensibility, and modularity. As the ecosystem matures, attention has turned to building confidential computing foundations, notably Trusted Execution Environments (TEEs) and secure enclaves, to support sensitive workloads. These efforts explore a variety of design directions, yet reveal important trade-offs. Some approaches achieve strong isolation guarantees, but fall short in scalability or broad adoption. Others introduce defenses, such as memory protection or side-channel resistance, although often with significant performance costs that limit deployment in constrained systems. Lightweight enclaves address embedded contexts, but lack the advanced security features demanded by complex applications. In addition, early-stage development, complex programming models, and limited real-world validation hinder their usability. This survey reviews the current landscape of RISC-V TEEs and secure enclaves, analyzing their architectural principles, strengths, and weaknesses. To the best of our knowledge, this is the first work to present such a consolidated view. Finally, we highlight open challenges and research opportunities, aiming toward establishing a cohesive and trustworthy RISC-V trusted computing ecosystem.

Keywords: RISC-V; security; trusted execution environment; confidential computing; secure enclave



Academic Editors: Shuwen Deng and Kun Yang

Received: 1 October 2025
Revised: 19 October 2025
Accepted: 21 October 2025
Published: 25 October 2025

Citation: Boubakri, M.; Zouari, B. A Survey of RISC-V Secure Enclaves and Trusted Execution Environments. *Electronics* **2025**, *14*, 4171. https://doi.org/10.3390/electronics14214171

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

The rise of heterogeneous and often untrusted platforms, from large-scale clouds to resource-constrained IoT devices, has elevated confidential computing to a primary concern for protecting data in use. Trusted Execution Environments (TEEs) [1] and secure enclaves have emerged as foundational building blocks to address this challenge, offering hardware-enforced isolation, attestable execution, and secure data management. Commercial implementations from major CPU vendors, including Intel Software Guard Extensions (SGX) [2], AMD Secure Encrypted Virtualization (SEV) [3], and ARM TrustZone [4], have demonstrated the practical viability of TEEs across diverse operational domains, from user-space application isolation to full virtual machine encryption.

However, the widespread academic and industrial adoption of these proprietary TEEs has revealed significant and inherent limitations. Each implementation only supports a narrow subset of the design space, making trade-offs that often result in critical constraints. For instance, Intel SGX [5] provides strong user-space isolation, but imposes severe memory limits and lacks secure I/O [6], while AMD SEV [7] (prior to SEV-SNP) offered memory encryption

without integrity protection, leaving it vulnerable to remapping attacks [8]. ARM TrustZone [9], although efficient for embedded systems, is constrained by a coarse-grained two-world model that lacks process-level isolation within the secure world [10]. These architectural compromises mean that no single proprietary TEE can adequately meet the diverse requirements of modern applications. Furthermore, their closed-source, vendor-locked nature inhibits independent verification, customization, and adaptation to novel threat models or use cases, stifling innovation and forcing developers to accept suboptimal compromises [11].

Security is another profound concern. Despite their architectural guarantees, proprietary TEEs have been consistently breached through microarchitectural side-channel attacks [12]. Vulnerabilities such as Foreshadow [13] and Load Value Injection (LVI) [14] against Intel SGX, or SEVered against AMD SEV [8], have demonstrated that memory encryption and logical isolation can be bypassed, leaking sensitive data from within enclaves. ARM TrustZone has also been shown vulnerable to attacks such as TruSpy [15] and TruSense [16], which exploit cache-based side channels between the Secure and Normal worlds, as well as CLKSCREW [17], which leverages power management features to induce faults in secure-world execution. These results illustrate that even TEEs based on coarse-grained isolation models are not immune to microarchitectural- or implementation-level attacks. These incidents underscore that TEE security is not merely a function of architectural design, but is deeply contingent on the underlying hardware's resilience to speculative execution and other microarchitectural leakage channels [18].

The limitations of proprietary TEEs have thus created a compelling need for open, customizable, and verifiable alternatives. It is in this context that the RISC-V instruction set architecture has gained prominence. Characterized by its modularity, extensibility, and transparency, RISC-V presents an ideal foundation for confidential and trusted computing research [19]. Its open nature allows for hardware security extensions to be rigorously evaluated, customized, and prototyped without vendor restrictions. Consequently, a vibrant landscape of academic and industrial initiatives has emerged, exploring a diverse array of TEE and secure enclave designs for RISC-V, each introducing distinct architectural innovations to address the shortcomings of their proprietary counterparts.

Despite this growing momentum and the proliferation of proposals, the research community lacks a consolidated understanding of the RISC-V TEEs and and secure enclaves design space. While some prior surveys have touched upon this topic [20–23], they typically offer only a limited overview. Broader surveys [21,22,24–27] on this area remain centered on proprietary architectures and overlook the unique opportunities and challenges inherent to an open hardware ecosystem [28].

To address this gap, this paper presents the first comprehensive survey of enclave and trusted execution in the RISC-V ecosystem. We systematically map the landscape of proposed TEE and secure enclave architectures, categorizing them based on their isolation mechanisms, threat models, and key design choices. Our analysis provides a structured comparison of these diverse approaches and identifies the recurring trade-offs that arise in their design. Furthermore, we highlight common limitations, discuss open research challenges, and outline promising directions for developing secure, efficient, and truly open trusted execution environments.

This survey includes analysis of ongoing standardization efforts within the RISC-V ecosystem, such as the Application Platform TEE (AP-TEE) initiative, and examines cutting-edge academic and industrial proposals published throughout 2025, providing a comprehensive view of the rapidly evolving RISC-V trusted computing landscape. By consolidating this fragmented landscape, this survey offers researchers and practitioners a critical reference point for advancing confidential computing on open architectures.

The main contributions of this paper are threefold:

(i) We provide a systematic survey of RISC-V TEEs and secure enclaves, examining their architectural foundations, security properties, and limitations.
(ii) We develop a taxonomy that highlights common design patterns and trade-offs across proposals.
(iii) We discuss open challenges and outline future research directions, aiming toward a cohesive and trustworthy trusted computing ecosystem for RISC-V. By consolidating a fragmented landscape, this survey offers both researchers and practitioners a structured reference to guide future work on secure and trustworthy open-hardware platforms.

2. Background

The purpose of this section is to establish the necessary foundations for the survey. We introduce the essential concepts of TEEs and secure enclaves, outlining their security properties, design goals, and common threat models. We then summarize the architectural elements of RISC-V that are directly relevant to confidential computing, including privilege levels, memory protection mechanisms, and recent security extensions. This background provides the terminology and taxonomy needed to analyze and categorize existing RISC-V TEE and secure enclave proposals in subsequent sections.

2.1. Trusted Execution Environments and Secure Enclaves

Trusted Execution Environment (TEE) is a term originally introduced by GlobalPlatform in its specifications [29]. Despite its formal definition, the term is often used loosely and is sometimes misappropriated for marketing purposes, resulting in a lack of consensus across industry and academia. The demand for stronger security features, driven by the complexity and connectivity of modern systems, from general-purpose computers to embedded and IoT devices, has accelerated the adoption of trusted computing concepts. Efforts such as [1] attempt to clarify the notion of a TEE by proposing a logical architecture and threat model. In this view, a TEE is a tamper-resistant environment that ensures (i) isolated execution of the code, (ii) the authenticity of the executed software, (iii) the integrity of the runtime state, (iv) the confidentiality of data and persistent storage, and (v) the ability to provide remote attestation to external verifiers.

The cornerstone of a TEE is isolation, commonly realized through a separation kernel or a comparable hardware-assisted mechanism. This allows for the coexistence of multiple execution contexts with different security levels, starting from hardware-enforced partitions. When partitions are isolated, the attack surface is reduced to the communication channels across them, which must be carefully designed to prevent leakage or privilege escalation. Separation kernels are typically lightweight to minimize the trusted computing base (TCB) and are built to uphold the four classical principles of separation: isolation, controlled information flow, fault containment, and resource independence.

A recurring question in the literature is whether the terms *TEE* and *secure enclave* are interchangeable. While related, they are not identical. A TEE is a broader concept describing the overall trusted execution environment with its security guarantees and interfaces, whereas a secure enclave is a specific realization of this concept that isolates the individual applications or processes within a protected region of execution. In practice, all enclaves are TEEs, but not all TEEs are enclaves: for example, ARM TrustZone provides a TEE based on a two-world model without process-level enclaves, whereas Intel SGX implements enclave-based TEEs.

2.2. RISC-V Security Landscape

In this subsection we outline the RISC-V features that are most commonly leveraged by enclave and TEE proposals, focusing on those that recur across the systems analyzed in

this survey rather than attempting to cover the full ISA. A number of architectural elements have become central to recent designs, particularly those related to privilege separation and memory isolation. At the core of the model are three privilege levels—user (U), supervisor (S), and machine (M)—which establish a hierarchical control structure for executing software with different trust assumptions. Physical Memory Protection (PMP) and its enhanced variant (ePMP) extend this model by enabling fine-grained control over memory regions and enforcing access permissions, forming the foundation for most RISC-V enclave designs. Virtual memory adds further flexibility by allowing for page-based isolation and translation for secure contexts. Additional extensions, including the hypervisor extension, cryptographic instructions, and memory tagging, strengthen RISC-V's ability to enforce isolation and mitigate certain classes of attacks. Together, these features constitute the architectural substrate on which RISC-V TEEs and secure enclaves are built. In the next section, we build on this foundation to develop a taxonomy of RISC-V TEE and secure enclave proposals and analyze how different designs leverage these mechanisms to achieve isolation, security, and trust.

2.3. Survey Methodology

This survey employs a systematic methodology to ensure comprehensive coverage of the RISC-V trusted execution landscape. The selection of architectures analyzed in Section 3 was governed by four principal criteria, designed to capture both architectural innovation and practical implementation maturity.

First, architectural significance was prioritized, focusing on proposals that introduced novel isolation mechanisms, security primitives, or execution models that substantially advance the state of trusted execution. Second, implementation fidelity required proof-of-concept realizations, whether on FPGA, simulation, or silicon, accompanied by empirical evaluation, ensuring that our analysis remains grounded in experimentally validated designs. Third, the diversity of design goals guided the inclusion of systems spanning embedded systems to cloud platforms, enabling the examination of performance-security-scalability trade-offs across the entire design space. Finally, community and industrial impact measures each proposal's influence on subsequent research and practice, including frameworks that have catalyzed academic follow-up work and emerging specifications reflecting industry convergence.

The survey synthesizes research from peer-reviewed publications, technical reports from industry and academic institutions, and publicly available documentation for open-source projects, covering the period from 2016 to 2025. Figure 1 illustrates the chronological evolution and comprehensive coverage of RISC-V TEE and secure enclave proposals analyzed in this survey. This inclusive methodological approach captures both the academic advancements and practical developments in RISC-V enclave and trusted execution.

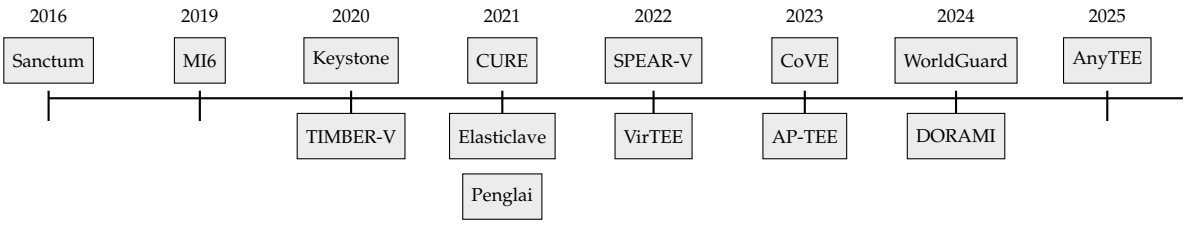


Figure 1. Chronological evolution of RISC-V TEE and secure enclave proposals (2016–2025).

3. Survey of RISC-V TEEs and Enclave Architectures

In recent years, a rich body of work has emerged in both academia and industry, exploring TEEs and secure enclaves for RISC-V. These proposals reflect diverse design goals, ranging from lightweight isolation for embedded systems to scalable enclave models

for cloud-class platforms. In this section, we provide a comprehensive survey of these architectures, summarizing their main design choices, strengths, and limitations.

To refine our comparative analysis, we introduce an Isolation Architecture dimension that classifies each system based on its primary mechanism for establishing security domains. This classification distinguishes between hardware-centric, software-centric, and hybrid approaches. Hardware-centric approaches rely on new or modified hardware primitives for isolation, such as custom memory management unit extensions or dedicated security coprocessors. Software-centric approaches enforce isolation primarily through software layers that leverage existing hardware features, such as standard PMP units. Hybrid approaches combine limited, targeted hardware changes with sophisticated software management to balance security and deployability.

To enable systematic comparison across these diverse approaches, we introduce a multi-layered taxonomy that categorizes TEE security mechanisms across three abstraction levels: Microarchitectural: Cache partitioning, speculative execution control. Hardware: Memory protection units, tagged memory. System-software: Security monitors, enclave runtimes. This layered approach allows for precise analysis of how different proposals address security challenges and where their trusted computing bases are established. The distinction between isolation architecture and security mechanism layers helps to clarify the fundamental trade-offs between security assurance, performance overhead, and hardware deployability analyzed in Section 4.1.

3.1. Keystone

Keystone [30] is designed as a flexible and extensible framework for TEEs, enabling developers to customize the TCB according to their specific security and performance needs. By leveraging the Physical Memory Protection (PMP) features of RISC-V, Keystone ensures robust memory isolation, allowing for enclaves to operate securely even in the presence of an untrusted operating system. Keystone is structured around three primary components: the Secure Monitor (SM), which operates in M-mode; the Enclave Runtime (RT), which functions in S-mode; and the enclave applications (EApps), which execute in U-mode.

The Secure Monitor (SM) plays a crucial role in the Keystone framework, as it is the sole component that operates in M-mode. Its primary responsibility is to maintain the separation between the OS and the enclave, ensuring that neither can interfere with the other. The SM achieves this by controlling access to physical memory, allowing for only the currently executing entity—either the host or the enclave—to access it. Furthermore, the SM enhances the Supervisor Binary Interface (SBI) to provide management services for enclaves to both the host and the enclave. The SM ensures a clean context switch by flushing enclave states, effectively preventing cache-based side-channel attacks through cache partitioning. Additionally, enclaves can be encrypted, and their page tables can be self-managed, rendering subtle attacks, such as controlled side-channel attacks, infeasible.

The runtime component is tasked with managing applications that run within the enclave (referred to as EApps). It serves as a supervisor for these applications, providing essential services such as system calls, memory management, and communication with the SM through the SBI. The reference implementation of the runtime is known as *Eyrie*, which delivers fundamental kernel functionalities. Developers have the option to modify Eyrie or create alternative runtime implementations to better suit the specific requirements of their EApps. The RT operates in S-mode within the enclave and is considered trusted by the enclave application, ensuring that it can effectively manage the enclave's operational needs while maintaining security. The enclave application (EApp) operates at the user level, executing sensitive application logic. Enclave applications are generally organized into two categories, regular code and sensitive code, ensuring that only the critical functions are executed within the secure enclave environment.

Another critical aspect of TEE deployment is the TCB, which should ideally be kept minimal. Keystone's TCB encompasses the entire SM, all M-mode code (including the bootloader and SBI implementation), and arguably the runtime. Although the runtime can be streamlined to provide only essential services to the EApp, the SM and other M-mode firmware cannot be significantly reduced. According to Lee et al. [30], Keystone's TCB consists of thousands of lines of code, in stark contrast to TrustZone's TCB, which comprises millions of lines. The hardware requirements for Keystone are minimal, necessitating only a standard RISC-V core, a secure method for storing device keys, and a secure bootloader. Notably, Keystone's open-source nature allows for greater transparency and adaptability.

According to Lee et al. [30], Keystone provides critical capabilities such as secure boot, remote attestation, and secure key provisioning to the Chain of Trust (CoT). The secure boot mechanism in Keystone lays the groundwork for a trusted system by verifying the integrity of the boot sequence. This is accomplished through either software or hardware implementations of a root of trust, which generates a new attestation key using a secure random source during the boot or reset phase. Each step of the boot process is validated by generating a hash and comparing it against a cryptographic signature. If any integrity verification fails, the boot process is halted, ensuring that the system remains secure only if the boot completes successfully. Measurement and attestation in Keystone are performed by the SM using the provisioned attestation key. During runtime, enclaves can request a signed attestation from the SM, which is then linked to a secure channel through a standardized protocol. Keystone provides further functionalities to meet the requirements of a TEE. It enables secure enclaves to access read-only timer registers maintained by the hardware via the `rdcycle` instruction. Additionally, it supports monotonic counters by preserving a minimal counter state within the SM memory. These features empower the SM to implement functionalities such as sealed storage, trusted timers, and rollback protection.

Sahita et al. [31] assert that the design of Keystone enclaves relies on contiguous memory allocation, which poses challenges for scalability in managing enclave memory after the boot process. Each enclave necessitates a dedicated PMP entry, allowing for the architecture to support up to N^2 enclaves, contingent on the availability of N hardware PMP registers. According to Dessouky et al. [32], Keystone's design enables it to defend against specific side-channel attacks that exploit shared resources, such as cache or memory structures. However, the separation of enclaves from the OS means that they are not treated as standard processes, necessitating the preservation and restoration of the OS state during enclave operations, which can introduce latency. Additionally, the requirement for a dedicated runtime for each enclave can lead to increased development complexity and redundancy, as multiple runtimes may implement similar functionalities. Dessouky et al. [32] also note that in Keystone, the enclave runtime can potentially incorporate device drivers to facilitate secure I/O operations with peripherals. However, the framework does not support a direct binding between enclaves and peripherals, which limits the ability of DMA-capable devices to securely interact with enclave memory. Consequently, this design choice leaves enclaves vulnerable to DMA attacks.

To enhance performance, Keystone implements way-based cache partitioning for shared L2 caches, assigning entire cache ways to the processor core's executing enclaves. This approach, while effective, may lead to inefficient cache usage, as cache lines not utilized by an enclave remain inaccessible to other software components. Furthermore, Dessouky et al. [32] emphasize that Keystone can mitigate certain hardware attacks, such as bus snooping, provided that the combined footprint of the enclave (EApp and runtime) and the SM fits entirely within on-chip scratchpad memory. Anh-Tien et al. [33] note that, despite the resilience of Keystone against certain attacks, the shared L1 cache of the untrusted operating system remains a potential vulnerability. Cache leakage may still occur following speculative execution, particularly if Keystone operates on an out-of-order processor like BOOM. This raises concerns regarding the potential for cross-address-

space Spectre attacks within the Keystone system. In such scenarios, the victim would be confined to the enclave, while the attacker would have unrestricted access to the broader environment.

Despite its strengths, the assumptions made in the design regarding the reliability of the SM, RT, and EApps raise concerns; the framework presumes these components are free from bugs, which may not hold true in practice. While the SM is designed to be small enough for formal verification, the RT's complexity could increase as more features are added, potentially introducing vulnerabilities. According to Donnini et al. [34], from a security perspective, Keystone does not provide comprehensive defenses against certain attack vectors, such as speculative execution and side-channel attacks, placing the onus of protection on developers. Additionally, the framework lacks robust defenses against side-channel attacks involving off-chip components, which could be addressed using techniques like Oblivious RAM. Lastly, the absence of non-interference guarantees for the SBI exposes the system to risks, including Iago attacks, as the RT can invoke untrusted system calls from the operating system, further complicating the security landscape for developers working with Keystone.

3.2. Sanctum

Sanctum [35] provides a framework for strong isolation of concurrently executing software modules, effectively protecting against a significant category of software attacks that exploit memory access patterns to extract sensitive information. Sanctum introduces isolated execution environments known as enclaves, designed to protect sensitive applications on RISC-V architectures. Each enclave operates at the user level and is paired with a non-sensitive application that invokes it.

Sanctum enforces enclave isolation through minimal hardware modifications to the Page Table Walker (PTW) within the Memory Management Unit (MMU). These changes ensure that the OS cannot access enclave memory and that enclaves are prevented from accessing OS memory or other enclaves by altering their page tables. The custom PTW is designed to block successful address translations for virtual memory addresses that do not correspond to the allowed physical memory addresses for the current execution context. The critical security functions of Sanctum are managed by a software component known as the Security Monitor (SM), which constitutes the system's TCB. The SM operates at the machine level of the RISC-V processor and undergoes verification during a secure boot process. While the operating system and its associated software are excluded from the TCB—loaded post-secure boot and not subjected to measurement—Sanctum enables the establishment and remote attestation of secure enclaves. This mechanism effectively initiates trust in additional software components, as the SM utilizes its cryptographic keys to measure and sign the secure enclaves. Importantly, the SM's authentication allows for remote parties to dismiss attestations from systems that have loaded known vulnerable versions of the monitor, thereby enhancing the overall security posture. Additionally, Sanctum's basic DMA protection is implemented through the inclusion of two registers in the memory controller.

The architecture incorporates a hardware True Random Number Generator (TRNG) and Physically Unclonable Functions (PUFs) to establish strong isolation through enclaves and defend against various software threats, including cache timing and passive address translation attacks. According to Anders et al. [21], Sanctum features minimal hardware modifications and has been successfully implemented on the Xilinx Zynq-7000 FPGA platform. Costan et al. [35] further note that Sanctum cores maintain the same clock speed as their non-secure counterparts, since there are no changes to the critical execution path of the CPU core. The architecture relies on the untrusted OS for managing enclave memory and providing essential services like interrupt handling and I/O operations. This reliance poses a risk, as a compromised OS could potentially execute controlled side-channel attacks against the enclaves. For instance, the adversary might deduce information about the enclave's internal

state by monitoring the enclave's page tables or by generating frequent interrupts. To counter these threats, Sanctum isolates the enclave's page tables within its memory and equips enclaves with the capability to detect and respond to unusual interrupt patterns.

Sanctum employs two primary strategies to mitigate cache side-channel attacks: first, it flushes sensitive processor resources, including the L1 cache and the Translation Lookaside Buffer (TLB), during every enclave context switch; second, it partitions the shared L2 cache using a memory page coloring technique, which allocates specific cache lines exclusively to enclaves. However, Dessouky et al. [32] affirm that the effectiveness of this cache partitioning is limited in practice, as it requires all software components to conform to the coloring scheme. Consequently, achieving this partitioning necessitates a complete rearrangement of the OS memory layout at runtime, which is often impractical.

Dessouky et al. [32] also assert that the enclaves in Sanctum consist of unprivileged user-level code, which restricts their ability to establish secure connections to peripherals such as sensors or GPUs that require privileged driver code. While Sanctum includes basic protections against DMA attacks by limiting access to a designated memory region, this feature is relatively rudimentary.

Wong et al. [36] stress that Sanctum's design is exclusively oriented towards mitigating software threats and does not extend its protective measures to physical attacks, which incur substantial hardware costs and performance penalties. Krentz and Voigt [37] emphasize that Sanctum necessitates resource-intensive attestation processes and does not prioritize the reduction of its communication latency. According to Cheang et al. [38], Sanctum does not provide formal verification for its hardware components. Nasahl et al. [39] further assert that Sanctum lacks architectural provisions for secure input/output operations, resulting in unprotected interactions with peripheral devices.

3.3. TIMBER-V

TIMBER-V [40] achieves memory tagging through modifications to the Memory Protection Unit (MPU), which enforces isolation between all processes and between the normal and trusted domains of the executing process, allowing for a clear separation of user mode (U-mode) and supervisor mode (S-mode) within both realms. The normal domains (N-domains) maintain the traditional U-mode and S-mode structure, permitting existing applications to operate without requiring modifications. Memory words in the N-domains are tagged with the *N* tag, while those in the trusted domains (T-domains) are assigned the *TU* and *TS* tags, corresponding to U-mode and S-mode, respectively. The architecture supports isolated execution environments, termed enclaves, within the trusted user mode (TU-mode), while the trusted supervisor mode (TS-mode) runs the TagRoot trust manager, which enhances the untrusted operating system with trusted functionalities. Additionally, TagRoot is responsible for enclave setup and offers essential services, including secure shared memory for communication with the normal domain, as well as functionalities for sealing and attestation. To protect against Direct Memory Access (DMA) attacks, additional tag engines are required for each peripheral with DMA capabilities.

Transitioning from N-domains to T-domains is facilitated through trusted callable entry points, marked with the TC tag. TIMBER-V employs a two-bit tagging system for each 32-bit memory word, allowing for four distinct tags. The architecture enforces strict rules for tag updates, permitting changes only within the same or lower security domains, thereby preventing privilege escalation. TS-mode and machine mode (M-mode) have unrestricted access to all tags, while TU-mode can only modify tags between N-tag and TU-tag, supporting dynamic interleaving of user memory. Notably, TU-mode is restricted from altering TC-tags, which are designated for secure entry points. However, these advanced features rely on specialized hardware enhancements, which may compromise the native execution of enclave

applications. According to Feng et al. [41], TIMBER-V introduces a significant performance overhead, averaging 25.2%, and does not address memory integrity protection.

The MPU in TIMBER-V enhances label isolation, effectively isolating each process while minimizing memory overhead. This design significantly mitigates memory fragmentation and promotes the dynamic reuse of untrusted memory across security boundaries. Beyond stack interleaving, TIMBER-V also enables innovative execution stack sharing across various security domains. The architecture is designed to be compatible with existing software and supports real-time operational constraints. A proof-of-concept implementation of TIMBER-V has been successfully evaluated using a RISC-V simulator [40].

Dessouky et al. [32] assert that while TIMBER-V enables fine-grained enclave creation for embedded systems, it currently lacks secure communication pathways between enclaves and external peripherals, such as sensors. Although it is theoretically possible to integrate drivers and services into the TagRoot to facilitate this communication, doing so would align TIMBER-V with high-level security models that have been criticized for significantly expanding the system's attack surface. Furthermore, TIMBER-V does not incorporate protective measures against cache side-channel attacks and remains susceptible to interrupt-based controlled side-channel attacks, as the handling of enclave interrupts is managed by the operating system.

3.4. MI6

MI6 [42] establishes a secure enclave framework characterized by robust microarchitectural isolation, ensuring that the enclave remains entirely segregated from the rest of the system at the microarchitectural level. Specifically, MI6 enhances the isolation guarantees of Sanctum by integrating hardware support into the RiscyOO out-of-order core, thereby extending its capabilities to address a broader range of threats, including side-channel and speculative execution attacks. To effectively counter these vulnerabilities, MI6 introduces a dedicated `purge` instruction designed to clear sensitive information from microarchitectural buffers and the L1 Data Cache prior to context switches. Furthermore, the operating system can only communicate with the enclave via the API managed by the Security Monitor (SM), a trusted software component operating at a higher privilege level. However, it is important to note that the design does not fully mitigate the D1 and D2 vulnerabilities, as these issues cannot be resolved solely through the flushing process.

MI6 implements hardware modifications to establish strong isolation through the spatial and temporal partitioning of resources. This approach involves allocating resources to protection domains without regard to their usage demands; for instance, last-level caches (LLCs) and DRAM bandwidth are divided among protection domains, with each domain limited to a specific fraction of the DRAM controller bandwidth, regardless of the memory usage of co-located domains. Additionally, MI6 shares similar limitations with Sanctum, as it employs an LLC partitioning strategy that does not scale effectively.

While MI6 ensures security, the functionality of the enclave is somewhat constrained, particularly due to a significant drawback: the absence of shared memory with external systems. Allowing for shared memory access with the operating system that would compromise the strong microarchitectural isolation between trusted and untrusted environments, potentially exposing the system to transient execution vulnerabilities such as Spectre [43], which leverage speculative execution paths in out-of-order processors to extract sensitive information through side channels. Consequently, the lack of memory sharing limits enclaves to performing isolated batch computations, restricting their ability to engage in external interactions and thereby narrowing the scope of potential applications.

According to Li et al. [44], MI6 is exclusively designed for the RiscyOO processor and lacks generalizability; it is dependent on the unique characteristics of the RiscyOO baseline processor and does not include mechanisms to effectively clear replacement tags in caches

and Translation Lookaside Buffers (TLBs). Furthermore, MI6 fails to address the cleaning of certain microarchitectural states (such as the issue queue), which could be vulnerable to emerging attack vectors. Also, the flushing process in MI6 does not ensure the writing back of dirty cache lines, rendering it incompatible with caches that utilize a write-back policy, which is prevalent in contemporary processors.

While MI6 utilizes both spatial and temporal isolation strategies, it does not address the verification challenges associated with formally proving isolation. The MI6 processor incorporates the `purge` instruction designed to clear microarchitectural state. Nevertheless, achieving comprehensive cleansing of all CPU states presents a significant challenge, heavily reliant on the intricate implementation specifics of the CPU.

In MI6, the overarching solution for the entire system can adversely impact the performance of standard applications with substantial memory demands. Frequent enclave executions necessitate barriers at each context switch, which can degrade the performance of regular applications, potentially deterring the adoption of enclaves. MI6 does not protect against adversaries executing concurrently on the same processor. The authors clarify that their "isolation mechanisms exclusively address software attacks" [42], and they do not provide defenses against denial-of-service (DoS) attacks. Furthermore, they explicitly state that they do not account for threats such as DRAM bit flipping.

3.5. HECTOR-V

In HECTOR-V [39], the design is based on two key innovations: the integration of a heterogeneous architecture that distinctly separates the Rich Execution Environment (REE) and Trusted Execution Environment (TEE) domains, and the introduction of a security-hardened RISC-V Secure Co-Processor (RVSCP) aimed at enhancing resilience against side-channel attacks (SCAs). HECTOR-V introduces a novel secure I/O path mechanism to manage device sharing and protect against unauthorized access. This mechanism employs an identifier-based strategy, where each device and processing core is assigned a unique identifier. This identifier is integrated into the communication system, allowing for fine-grained access control. Each transaction is validated by the Security Monitor (SM) module based on these IDs, rejecting any unauthorized access attempts. The core processor ID is hard-coded in hardware, while process and peripheral IDs are assigned dynamically at runtime. This hard-coding ensures that attackers cannot alter the core IDs.

In HECTOR-V, managing access permissions is done using a hardware-based Security Monitor. Only one SM owner is permitted at any given time, and this owner is responsible for defining access rights to resources. The secure boot process is executed by granting exclusive access to the secure storage to the first virtual core of the TEE (VC0). This access right is permanently hard-coded and solely owned by VC0, while other virtual TEE cores can only retrieve code from claimable Block RAM (BRAM). Upon reset, the reset unit designates VC0 as the SM owner, keeping the REE processors in a halted state. VC0 then executes the Zero Stage BootLoader (ZSBL) from the secure storage, initiating the first authentication of the system, which serves as the Root of Trust (RoT) for HECTOR-V. Following this, the ZSBL configures the Memory Protection Unit (MPU) for external memory access rights, including those for the Secure Digital (SD-card) and Double Data Rate (DDR) memory. VC0 subsequently verifies the hash value of the Berkeley BootLoader (BBL) against the expected value stored in secure storage. If the verification is successful, the BBL is loaded into the main memory, and VC0 releases the SD-card driver along with the claimed DDR memory regions. Finally, VC0 transfers SM ownership to the REE processors and triggers the reset unit to initiate the REE. However, since the secure boot program remains accessible from the RVSCP and both the REE and TEE share the same system-on-chip (SoC), there exists a potential risk of exposing the RoT to vulnerabilities from the REE side, despite the implementation of secure storage elements. The architecture does

not incorporate dedicated hardware computation units for cryptographic algorithms, which may result in a less efficient secure boot process. This absence of specialized cryptographic support could lead to increased latency during the secure boot sequence, as the system relies on general-purpose processing capabilities rather than optimized hardware acceleration for cryptographic operations. Consequently, this design choice may impact the overall performance and responsiveness of the secure boot mechanism within the HECTOR-V framework.

The authors argue that the duplication of resources in HECTOR-V effectively mitigates cache and microarchitectural side-channel attacks by ensuring that sensitive components are not shared between the secure and non-secure domains. However, they acknowledge that this approach may not be entirely feasible in real-world applications where resource utilization constraints are significant. Additionally, HECTOR-V does not support the integration of programmable resources, which are crucial for SoC-FPGAs. Furthermore, while HECTOR-V provides a dedicated processor for secure applications, it lacks the capability to create multiple secure domains, limiting its flexibility in complex security scenarios.

3.6. CURE

CURE [45] is a robust TEE architecture that leverages innovative hardware security primitives within the RISC-V framework. This architecture supports the coexistence of various enclave types, including kernel-space, user-space, and sub-space enclaves, within a single system. To facilitate this, CURE incorporates three key hardware enhancements: dedicated core registers for monitoring enclave execution, a system bus arbiter to manage access control for bus transactions, and a mechanism for partitioning the shared cache. These enhancements collectively bolster the isolation of enclaves and provide defenses against side-channel attacks (SCAs). The Security Monitor (SM) in CURE functions as a sub-level enclave. A significant benefit of implementing the SM as a sub-level enclave is the substantial reduction in the system's Trusted Computing Base (TCB), as it excludes all non-security-related code at the machine level. This flexibility allows for developers to select the enclave type that best aligns with the requirements of their sensitive applications without needing to modify the application to fit a specific enclave model.

The user- and supervisor-level enclaves in CURE can integrate device drivers into their execution environment. Coupled with CURE's hardware security features, this capability facilitates the exclusive assignment of peripherals to specific enclaves, known as enclave-to-peripheral binding. The system utilizes enclave IDs stored in the core registers, which are propagated throughout the architecture to track the active enclave on each core. These IDs are established during the enclave's initialization, termination, and context-switching processes. The bus arbiter evaluates access permissions based on the enclave ID for each memory access request, redirecting any unauthorized transactions to a designated area, thereby preventing their execution. In terms of enclave-to-peripheral interactions, CURE ensures that memory access is strictly regulated, eliminating the need for encryption or authentication in communications between enclaves and peripherals. To address SCAs, CURE employs two primary strategies: flushing the L1 cache and implementing a way-based partitioning approach for the L2 cache. However, Stapf et al. [46] note that the allocation of entire cache ways to enclaves may result in suboptimal cache utilization due to the coarse granularity of cache ways.

A pivotal hardware security feature of CURE is the filter engine integrated into the system bus. This filter engine serves dual purposes: it allows for the exclusive assignment of memory regions to enclaves and establishes access controls for peripherals, determining which enclaves can communicate with them via Memory-Mapped I/O (MMIO). Furthermore, Stapf et al. [46] point out that the filter engine incorporates registers and control logic for all Direct Memory Access (DMA)-capable devices, restricting their access to designated

memory areas. This mechanism facilitates secure communication between enclaves and DMA-capable devices without requiring encryption.

CURE's software TCB, represented by the SM, is designed to be minimal, encompassing only the security-critical code while omitting the standard firmware code typically present at the machine level. The SM is responsible for managing enclaves and executing all security-sensitive operations, such as enclave binary verification, key management, and persistent storage of enclave states. However, Kuhne et al. [47] aver that CURE necessitates hardware modifications to implement security features, including the introduction of a filter engine that enforces access controls at the system bus level and the integration of a unique enclave identifier within the CPU architecture to manage enclave operations effectively.

According to Schneider et al. [48], a notable limitation of CURE is that its attestation mechanisms do not encompass peripheral devices, which could expose vulnerabilities in the communication between enclaves and hardware components. Furthermore, the design of kernel-space enclaves within CURE mandates that they operate on dedicated CPU cores, which, as far as current knowledge indicates, do not have the capability to relinquish control back to the operating system. This design choice may lead to inefficient resource utilization, as these cores remain idle while awaiting data from peripheral devices, potentially hindering overall system performance [48]. Regarding the Root of Trust (RoT), CURE does not establish a RoT mechanism, but presumes that the secure boot sequence is completed upon system reset, with the initial bootloader in ROM responsible for verifying and loading the firmware, including the Secure Monitor (SM), into the appropriate Random Access Memory (RAM). Consequently, Kieu-Do-Nguyen et al. [49] assert that in terms of RoT-based secure boot processes, CURE does not offer any innovative solutions beyond the traditional reliance on hard-coded keys stored in ROM, which may limit its adaptability to more dynamic security requirements. Additionally, Chen et al. [50] emphasize that CURE lacks dedicated hardware support for accelerating cryptographic algorithms, which results in reduced efficiency for encryption and decryption processes.

3.7. CoVE

CoVE [31] serves as a foundational architecture for confidential computing tailored for RISC-V platforms, with its secure execution environment referred to as a TEE Virtual Machine (TVM). Central to this architecture is the TEE Security Manager (TSM) driver, which operates in M-mode—the highest privilege level in RISC-V—facilitating transitions between confidential and non-confidential operational contexts. The TSM driver is responsible for managing memory page allocations to TVMs via the Memory Tracking Table (MTT), ensuring proper isolation and security. It also plays a crucial role in measuring and initializing the TSM, which acts as a trusted intermediary between the hypervisor and the TVMs. The CoVE Trusted Computing Base (TCB) is primarily composed of the TSM, which serves as the intermediary for security enforcement between Trusted Execution Environments (TEEs) and non-TEE elements, alongside hardware components that uphold the confidentiality and integrity of data in use. Consistent with other frameworks, the hypervisor remains untrusted, tasked with the management of resources across all workloads, encompassing both confidential and non-confidential applications. CoVE specifies an Application Binary Interface (ABI) that allows for the hypervisor to invoke virtual machine management functions from the TSM.

The architecture employs a multi-layered attestation framework, starting from the hardware and extending through the TSM driver, TSM, and TVM. Each layer undergoes a process of loading, measurement, and certification by its predecessor, establishing a robust chain of trust for system integrity verification. The TVM can request a certificate from the TSM, which includes attestation evidence linked back to the hardware, thereby providing

a reliable method for confirming the authenticity of the TVM and its operating software. CoVE employs cryptographic mechanisms to protect confidential memory against physical access, ensuring confidentiality, integrity, and replay protection. This includes the use of separate cryptographic keys for different TVM workloads to enhance security.

While CoVE aims to minimize the TCB, the introduction of new hardware primitives and ISA extensions increases the complexity of the overall system design and implementation. Although the architecture is designed for performance, the additional layers of isolation and security mechanisms introduce some overhead, particularly in scenarios with frequent context switching or resource allocation. The authors note that a common drawback of existing approaches, including CoVE, is the lack of support for confidential I/O, which remains an ongoing area of work [31].

3.8. WorldGuard

The RISC-V WorldGuard (WG) [51] architecture enhances isolation by implementing a comprehensive system-wide approach through the concept of *Worlds*, which serve as distinct execution contexts encompassing both agents (components that initiate transactions) and resources (components that respond to transactions). Each World is uniquely identified by a hardware World Identifier (WID), with the total number of unique WIDs being platform-specific, defined by the parameter `NWorlds`, and limited to a maximum of 32 Worlds. The WG architecture is designed to facilitate the static allocation of agents and resources to Worlds, typically managed by M-mode firmware or a Trusted Execution Environment (TEE) during the system boot process. WorldGuard allows for the dynamic transfer of Security Monitor (SM) ownership among different parties, facilitating a broader range of use cases. Nasahl et al. [39] posit that WorldGuard's architecture emphasizes a generalized approach to managing access permissions. This flexibility in ownership and access management positions WorldGuard as a versatile solution for creating secure execution environments within heterogeneous architectures.

WorldGuard integrates Physical Memory Protection (PMP) and Physical Memory Attributes (PMA) within the RISC-V Instruction Set Architecture (ISA). Hoang et al. [52] assert that WorldGuard operates with shared processors for both the TEE and Rich Execution Environment (REE). This architecture is designed to provide a more robust separation of execution contexts, thereby mitigating potential security risks. However, it is important to note that WorldGuard does not focus on optimizing the secure boot process; instead, it aims to augment the existing TEE models. As such, it employs a traditional boot flow for secure initialization, relying on a bootloader that contains hard-coded root keys stored in Read-Only Memory (ROM). This bootloader is the first component executed, tasked with verifying and loading the secure channel into the main memory, ensuring that both the boot program and the Root of Trust (RoT) remain within the TEE domain. Consequently, Hoang et al. [52] note that while the RoT and boot program remain within the TEE domain, the potential for attack persists. Although WorldGuard's bootloader program is accessible, details regarding its hardware implementation remain undisclosed. Pinto et al. [53] emphasize that while WorldGuard enhances isolation, it still retains vulnerabilities to conventional software-based side-channel attacks, as the foundational elements of the secure boot process are not inherently fortified against such threats. Notably, the specification does not encompass efficient mechanisms for dynamic reconfiguration of Worlds while the system is operational, which remains outside its current scope.

In contrast to the RISC-V (e)PMP, which enforces access control through a set of memory configuration rules directly at the hart level, WG adopts a more flexible approach. It does not prescribe a specific method for the propagation and verification of WIDs across the platform or bus, leaving these implementations to be platform-specific. Consequently,

various platforms may adopt different strategies for checking WIDs, with bus fabrics implementing support for WIDs in ways that are tailored to their specific architectures.

3.9. SPEAR-V

SPEAR-V [54] employs a lightweight memory tagging mechanism to enforce fine-grained access control for enclave memory. Unlike some existing architectures that necessitate extensive modifications to the operating system, SPEAR-V integrates seamlessly with unmodified OS-managed paging structures, allowing for efficient memory management. The architecture utilizes 24-bit memory tags, which are sufficient for its security requirements while minimizing overhead.

SPEAR-V effectively mitigates same-core side-channel attacks such as branch shadowing, but it does not specifically address cross-core side-channel vulnerabilities, which may limit its effectiveness in multi-core environments. Additionally, the architecture's reliance on a single-core processor means that it does not consider the complexities introduced by multi-core systems in its baseline design. While SPEAR-V does not specifically target physical threats such as memory-bus snooping or malicious DRAM modifications, it allows for the integration of orthogonal techniques such as memory encryption to enhance security against certain physical vulnerabilities. This indicates a flexible approach to security, albeit with the acknowledgment that physical attack defenses are outside the primary scope of the architecture.

SPEAR-V also supports dynamic memory allocation and arbitrary nesting of enclaves, enhancing its scalability and flexibility for various applications. However, despite its robust design, the architecture does not provide explicit defenses against memory corruption or code reuse attacks, which could still pose risks if vulnerabilities exist in the enclave code.

3.10. VirTEE

VirTEE [55] is built on a secure platform infrastructure that divides physical memory into enclaves, utilizing specific registers for memory management instead of traditional page tables. This innovative approach allows for the support of large enclave sizes and efficient memory access control, leveraging the RISC-V hypervisor extension to facilitate virtualization. VirTEE enhances security by implementing cache partitioning, ensuring that each CPU core executing an enclave has its own last-level cache that is not shared with other cores. This design mitigates the risk of sensitive information leakage through cache timing attacks.

The architecture provides strong physical enclave memory isolation, protecting enclave memory from unauthorized access by malicious software, including the operating system. VirTEE's ability to support large enclave sizes is beneficial for running multiple virtual machines and applications simultaneously without significant overhead. The architecture demonstrates moderate performance, as evaluations indicate that it incurs only a modest performance overhead on standard benchmarks and real-world applications, making it suitable for practical deployment in cloud environments.

However, VirTEE also has its limitations. It is heavily dependent on specific hardware configurations, which may restrict its deployment in environments lacking the necessary infrastructure. Additionally, while it offers resilience against side-channel attacks, it does not protect enclaves from memory corruption attacks, leaving a potential vulnerability. Furthermore, the architecture assumes that peripherals, such as hard drives, are accessible to adversaries, which could lead to data leaks. Lastly, denial-of-service (DoS) attacks are outside the scope of VirTEE's guarantees, as most TEE architectures do not provide assurances regarding availability.

3.11. DORAMI

DORAMI [47] leverages the enhanced Physical Memory Protection (ePMP) feature of RISC-V, allowing for fine-grained control over memory access and isolation between different execution modes (P, F, S/U). This design minimizes the need for significant hard-

ware changes, facilitating easier adoption in existing systems. However, the architecture's effectiveness is contingent on the availability of ePMP support, which is not present in all RISC-V platforms, thus limiting its applicability. DORAMI enforces strict intra-mode isolation between the Security Monitor (SM) and firmware, which helps to mitigate the risks associated with shared resource exploitation. By restricting access to PMP registers solely to the SM, the architecture reduces the potential for side-channel attacks that could manipulate memory protection settings. Nonetheless, the complexity of managing PMP configurations still introduces vulnerabilities when not handled properly.

DORAMI's reliance on hardware-based memory isolation via PMP provides a layer of defense against unauthorized access or manipulation of memory regions. The compartmentalization of the SM from the firmware further limits the impact of physical attacks on the firmware. However, the architecture primarily focuses on memory isolation and does not comprehensively address all aspects of physical security, such as tamper resistance. DORAMI is adaptable to various RISC-V platforms, including those with only standard PMP support, enhancing its scalability across different hardware configurations. It is designed to manage multiple enclaves, allowing for deployment in diverse security environments. However, as the number of compartments and enclaves increases, managing their interactions and ensuring that security becomes complex, potentially impacting scalability.

DORAMI aims to achieve its security goals with minimal performance penalties, ensuring that applications running in the RISC-V environment do not experience significant slowdowns. The architecture modifies PMP configurations during context switches to maintain performance while enforcing security. However, the implementation on certain platforms, such as Rocketchip, has shown performance variability, indicating that further optimizations are needed to ensure consistent performance across different environments. Additionally, while designed to minimize overhead, the modifications during context switches introduce latency, particularly in high-frequency switching scenarios.

3.12. Elasticlave

Elasticlave [56] enhances the enclave architecture by enabling each enclave to manage multiple physical memory regions that can be selectively shared with other enclaves. This design allows for an enclave to request access to another enclave's memory regions, which the owner can grant, thereby facilitating more efficient inter-enclave communication without the overhead of data copying or encryption. Elasticlave can be implemented on RISC-V and is designed to maintain a relatively simple hardware complexity, requiring only a privileged Security Monitor (SM) that spans approximately 7000 lines of code. This simplicity contrasts with traditional TEEs, which often necessitate more complex hardware setups. The architecture is also adaptable, as it can be integrated into various TEE implementations, including Intel SGX and ARM TrustZone, although this may involve specific changes to accommodate different memory management models.

Elasticlave permits an enclave to dynamically adjust permissions for shared memory regions, allowing for other enclaves to write to these regions. However, the authors highlight that this flexibility necessitates careful management of write permissions to mitigate potential interference between enclaves, ensuring that data integrity is maintained during concurrent access. The architecture requires enclave programs to invoke a `map` operation to access shared memory regions, which must then be approved by the owner through a `share` operation. Kuhne et al. [47] state that this explicit permission model enhances security, but also adds complexity to the programming model. Feng et al. [41] maintain that this sharing mechanism lacks flexibility. Specifically, the enclave must identify the target enclave by its ID, which requires that the other enclave be created beforehand and that its ID be communicated to the sharing enclave. Additionally, the shared data is tightly coupled to

the lifecycle of the enclave; if the sharing enclave terminates, the data is lost. Elasticlave introduces a new `share` operation that allows for an enclave to designate a contiguous range of virtual memory addresses for sharing with another enclave identified by a specific ID. To minimize the performance costs associated with data transfer, Elasticlave assumes a common encryption key across all enclaves. However, Feng et al. [41] claim that this assumption raises security concerns, as it undermines the cryptographic separation typically maintained between enclaves. Furthermore, they argue that this design is not compatible with existing TEE frameworks, such as AMD SEV and Intel TDX, which rely on the principle that each enclave utilizes its own unique encryption key. According to Feng et al. [41], the rigidity of Elasticlave's sharing model leads to several limitations: the sharing enclave must be aware of the target enclave's ID, the target enclave must be instantiated prior to the sharing operation, and the shared data is rendered inaccessible if the sharing enclave is terminated.

Pan et al. [57] assert that Elasticlave enhances the efficiency of memory sharing between enclaves, but faces challenges in scalability due to the fixed number of Physical Memory Protection (PMP) registers, which restricts the number of simultaneously protected memory regions. They argue that while Elasticlave allows for improved performance in data-sharing workloads, its reliance on the RISC-V PMP architecture limits the number of enclaves that can operate concurrently, potentially impacting its effectiveness in high-demand cloud environments. The architecture's design facilitates inter-enclave communication through a shared memory model, yet the constraints imposed by the RISC-V PMP mean that Elasticlave may not adequately support applications requiring extensive concurrent enclave interactions. Pan et al. [57] emphasize that, due to the limitations of the RISC-V specification, Elasticlave can only manage a limited number of memory regions, which restricts its ability to scale effectively in scenarios with numerous consumer enclaves.

Yu et al. [56] acknowledge that, while Elasticlave focuses on defining a memory interface, it does not specifically address microarchitectural implementation flaws or side-channel vulnerabilities. Elasticlave does not directly address defenses against attacks on physical RAM or bus interfaces, indicating that such protections are considered orthogonal to the architecture's primary focus. Kuhne et al. [47] further discuss that, while Elasticlave enables the mapping of shareable physical memory regions to an enclave's virtual address space, there are notable drawbacks. First, developers must explicitly identify which portions of their applications are shareable, often necessitating significant code restructuring to isolate these components into separate enclave memory. Second, the dynamic mapping and unmapping of memory regions require local attestation, which ensures that the newly mapped memory is in a secure and expected state. This reliance on attestation complicates the measurement properties of applications, as it ties the security guarantees of a program to the integrity of multiple physical memory regions.

3.13. Cerberus

Cerberus [58] utilizes a formal verification framework to enhance the security and efficiency of enclave memory sharing. Cerberus is implemented on the RISC-V Keystone platform, which necessitates specific support for features like Physical Memory Protection (PMP) to ensure memory isolation. This reliance on particular hardware capabilities limits its applicability to platforms that do not support these features. While the authors emphasize the formal verification of the Secure Remote Execution (SRE) property, Cerberus does not provide extensive details on specific countermeasures against side-channel vulnerabilities, which remain a critical concern in enclave designs.

The architecture introduces two critical operations: `Snapshot` and `Clone`. The `Snapshot` operation effectively transforms the executing enclave into a read-only entity, while the `Clone` operation allows for the creation of a child enclave that can access the same memory contents as

its parent at the moment of cloning. To ensure consistent functionality, the virtual address spaces of both the parent and child enclaves must align immediately following the `Clone` operation. However, any write operations performed by the child enclave will lead to divergence from the shared memory, as these actions trigger a copy-on-write mechanism that allocates new memory for modifications. This presents a limitation for Cerberus, as the advantages of memory sharing may diminish over time as the child enclave's memory diverges from the original snapshot. Nevertheless, Cerberus proves to be particularly effective in scenarios where enclaves primarily write to a limited portion of memory while sharing the remainder. It is the responsibility of the programmer to strategically determine when to invoke the `Snapshot` operation.

The Cerberus interface is designed to integrate seamlessly with process-creation system calls, thereby reducing the startup latency associated with enclave initialization. By providing a programmable interface, Cerberus enhances the overall efficiency and responsiveness of server enclave applications, ultimately improving end-to-end latency for various use cases. Cerberus also aims to protect against physical attacks by maintaining a strong memory isolation model, but the effectiveness of this protection is contingent on the underlying hardware's security features. Scalability is addressed through the introduction of a single-sharing model with read-only shared memory, which simplifies the verification process and allows for efficient memory sharing across multiple enclaves. This design choice enhances performance by reducing initialization latency and minimizing computational overhead during enclave operations. However, the limitation lies in the fact that the sharing model restricts each enclave to access only one read-only shared memory, which is not suitable for all use cases requiring more complex memory-sharing scenarios.

3.14. AP-TEE

AP-TEE (Application Platform Trusted Execution Environment) [59] is a draft specification being developed within the RISC-V Security and Confidential Computing Working Group. AP-TEE leverages the RISC-V virtualization extensions to enable confidential virtual machines (VMs) within a TEE. Its design goal is to provide a RISC-V counterpart to confidential computing frameworks such as AMD SEV, Intel TDX, and Arm CCA. As the AP-TEE specification is not yet finalized, the description here reflects the preliminary information available in the literature [59], and may evolve as the standard matures.

In AP-TEE, virtualization is divided into two domains: *Non-Confidential*, where a conventional operating system executes, and *Confidential*, which remains isolated from the former. The TSM Driver, executing in M-mode, serves as the relay between the two domains. The TEE Security Manager (TSM), which runs in HS-mode within the Confidential domain, operates as a passive software component that processes requests from both the hypervisor and the TEE Virtual Machines (TVMs).

Two types of Application Binary Interfaces (ABIs) are defined for TSMs. The first is the *TH-ABI* (TEE-Host ABI), which governs the interaction between the non-confidential hypervisor and the TSM. It supports operations such as TVM creation, memory page management, and execution scheduling. The second is the *TG-ABI* (TEE-Guest ABI), which defines the interface between TSM and a TVM. This ABI provides support for attestation, I/O operations, and memory management.

Unlike other RISC-V TEEs that rely on Physical Memory Protection (PMP), AP-TEE introduces the Memory Tracking Table (MTT) to distinguish between confidential and non-confidential virtual memory pages. This mechanism enables the precise identification and enforcement of isolation at the granularity of individual pages.

The AP-TEE boot sequence begins with the execution of the TSM Driver in M-mode. During this phase, the startup of the TSM Driver is measured, and its hash value is recorded in the hardware Root of Trust (RoT) for attestation purposes. Once initialized,

the TSM executes in HS-mode within the Confidential domain. Both its startup and state are measured and recorded. Subsequently, the hypervisor and host OS are launched in HS-mode within the Non-Confidential domain.

To create a TEE Virtual Machine (TVM), the host OS and hypervisor interact with the Trusted Host ABI (TH-ABI). This process allocates memory within the confidential domain, initializes the virtual CPU (vCPU), and launches the TVM. Each stage—TSM Driver, TSM, and TVM—is measured and recorded, forming a chain of trust that underpins the attestation process. In other words, attestation in AP-TEE covers the entire boot sequence: *TSM Driver* → *TSM* → *TVM*.

3.15. Penglai

Penglai [60] is an enclave system that adopts a hardware–software co-design to overcome scalability limitations present in prior trusted execution environments. The architecture introduces two central primitives: the *Guarded Page Table (GPT)* and the *Mountable Merkle Tree (MMT)*. The GPT enables fine-grained memory isolation at the page level by protecting host page tables within a restricted memory region, ensuring that only authorized entities can access or modify them. The MMT serves as an integrity protection structure, supporting on-demand, scalable memory encryption through a mountable hash forest. Together, these primitives allow for Penglai to dynamically manage secure memory, achieving support for thousands of concurrent enclaves and scaling up to 512 GB with low runtime overhead.

Beyond its memory model, Penglai introduces *shadow enclaves* and a *fork-style enclave creation mechanism*, which significantly reduce startup latency. Reported evaluations demonstrate reductions of up to three orders of magnitude compared to traditional enclave initialization, making the architecture well suited for short-lived cloud functions and serverless workloads. Performance measurements further indicate modest overheads—approximately 5% for memory-intensive applications—while maintaining strong security guarantees against both software-based and physical attacks. These properties position Penglai as a scalable alternative to contemporary enclave architectures, addressing a broader set of use cases in cloud and edge computing.

3.16. AnyTEE

AnyTEE [61] is an open and interoperable framework for building Software-Defined Trusted Execution Environments (sdTEEs) that aim to address the fragmentation and compatibility challenges prevalent in contemporary TEE technologies. By leveraging widely available hardware virtualization extensions, AnyTEE enables the emulation and customization of diverse TEE models—such as Intel SGX and Arm TrustZone—across multiple Instruction Set Architectures (ISAs), including Arm and RISC-V.

The framework introduces a hierarchical execution model that supports nesting, composition, and fine-grained access control through nested page tables, enabling sdTEEs to coexist and interoperate on the same platform. Key innovations include configurable security policies, support for unmodified trusted applications, and enhanced isolation mechanisms such as intra-privilege memory protection.

Evaluations demonstrate that AnyTEE achieves near-native performance, with overheads below 3%, while providing strong security guarantees against software-based attacks. The framework is implemented as an open-source system, offering a portable and extensible foundation for future TEE architectures.

To systematically contrast the diverse approaches to enclave and trusted execution on RISC-V, we provide a comprehensive comparison of all surveyed systems in Table 1. The table highlights critical differences in isolation architecture, security properties, and implementation specifics.

Table 1. Comparison of RISC-V TEE architectures across design, security, and implementation criteria.

System	Iso. Arch.	Enclave Type	Priv.	Memory Isolation	Sec. I/O	Cache SCA	HW Mod.	Impl. Valid.	SDK	TCB Size	Crypto.	Compliance
Keystone [30]	SW	Custom Runtime	U + S	PMP	○	◐	●	FPGA	Partial	∼8.4 k LoC	○	○
Sanctum [35]	HW	User Enclave	U	PMP + PTW	○	●	◐	FPGA	None	∼5 k LoC	○	○
TIMBER-V [40]	HW	Sub-process	U	Tagged	○	◐	○	Simulator	None	∼3 k LoC	○	○
CURE [45]	HW	Multi-level	U/S/M	Cache-tag	●	●	○	FPGA	None	Partial SM	○	○
Cerberus [58]	SW	Custom PMP	U + S	PMP	○	◐	●	FPGA	None	∼7 k LoC	○	○
CoVE [31]	Bal	TVM-based	S	MTT + G	●	●	◐	FPGA	Partial	∼2 k LoC	○	◐
Dorami [47]	SW	Firmware	U	PMP	○	○	●	FPGA	None	n/a	○	○
Elasticlave [62]	SW	Temporal	U	PMP + Shared	○	○	●	FPGA	None	∼8.5 k LoC	○	○
Hector-V [39]	HW	Core-part	Core	Interconnect	●	●	○	FPGA	None	HW SM	◐	○
MI6 [42]	HW	Microarch.	U	Cache Part.	○	○	○	FPGA	None	Unknown	◐	○
SPEAR-V [54]	SW	S-mode	S	PMP	○	○	●	Simulator	None	Unknown	○	○
WorldGuard [51]	HW	Domain	Core	Bus Filter	●	●	○	FPGA	None	SoC-based	○	○
Penglai [60]	Bal	User Enclave	U	MMU	○	◐	◐	FPGA	None	∼10.2 k LoC	○	○
AnyTEE [61]	SW	sdTZ/sdSGX	U + S	Page Tables	◐	◐	●	FPGA	None	∼9.4 k LoC	○	○

System: TEE implementation name. **Iso. Arch.**: Isolation Architecture (HW: Hardware-reliant; SW: Software-reliant; Bal: Balanced). **Enclave type**: Isolation type. **Priv. Levels**: Privilege levels used (U: User; S: Supervisor; M: Machine, Core). **Memory Isolation**: Mechanism for memory separation. **Sec. I/O**: Support for secure I/O access. **Cache SCA**: Side-channel attack protection. **HW Mod.**: Hardware modification level (● = None; ◐ = Minor; ○ = Structural). **Impl. Valid.**: Primary validation platform (FPGA, Simulator, ASIC). **SDK**: Availability of development toolchain. **TCB size**: Trusted code base size. **Crypto.**: Support for secure/accelerated cryptography. **Compliance**: Standards conformance. Symbol key: ● = full support; ◐ = partial; ○ = none. **n/a:** Not applicable or not available.

4. Discussion

The survey of RISC-V Trusted Execution Environments (TEEs) and secure enclaves reveals a vibrant but fragmented ecosystem. Each proposal addresses particular limitations of proprietary TEEs—such as closed design, rigid threat models, and vendor lock-in—yet none offers a comprehensive or universally deployable solution. Instead, common design trade-offs and systemic challenges emerge across the body of work.

4.1. Design Trade-Offs

Lightweight approaches such as DORAMI or SPEAR-V emphasize minimal hardware changes and low performance overhead, but provide only partial defenses against side-channel or physical attacks. In contrast, frameworks like HECTOR-V or CURE integrate extensive hardware modifications to strengthen isolation and enable peripheral binding, at the cost of higher complexity and reduced portability.

Different proposals also illustrate distinct models of privilege separation. Sanctum resembles Intel SGX, supporting user-level enclaves, but relying on the OS for privileged functions. TIMBER-V instead provides a trusted supervisor mode, allowing for privileged services inside enclaves. Keystone combines these approaches by introducing a minimal enclave runtime in S-mode, while still delegating some system services to the untrusted OS. CURE extends this model further by supporting multi-level enclaves, including deep enclaves in M-mode that isolate critical machine-level code. Penglai departs more radically, adopting a microkernel-like approach where service-oriented enclaves provide system functionalities.

Beyond privilege models, our comparative table highlights the trade-offs between TCB size, SDK availability, and compliance. Sanctum maintains a minimal $\sim$5 k LoC TCB, while Penglai and Elasticlave reach over 10 k LoC, raising the verification burden. Keystone and AnyTEE expose partial SDKs, but no system yet integrates GlobalPlatform APIs or PKCS#11, limiting compatibility with existing trusted applications. Similarly, compliance remains absent across all surveyed works, underscoring the distance from industrial certification.

The introduced Isolation Enforcement dimension further illuminates the core trade-offs. Hardware-centric approaches (e.g., Sanctum, CURE, HECTOR-V) generally provide stronger isolation guarantees and lower runtime performance overhead by pushing security checks into the hardware. However, this comes at the cost of reduced portability and higher barriers to adoption due to the need for non-standard hardware. Conversely, software-centric approaches (e.g., Keystone, Elasticlave, DORAMI) prioritize deployability and flexibility, operating on available hardware, but often incur higher runtime overhead for context switching and monitoring and may offer weaker guarantees against sophisticated hardware attacks. Hybrid approaches attempt to balance these extremes, but can inherit complexities from both domains. This enforcement strategy is a primary factor influencing the TCB size, performance profile, and, ultimately, the applicable domain for each proposal.

4.2. Scalability and Serverless Use Cases

A recurring limitation of enclave systems is scalability. Keystone and CURE are constrained by the number of PMP entries, limiting the number of concurrent enclaves. Sanctum, likewise, supports only a modest number of enclaves, due to its reliance on page table modifications. Penglai addresses this gap by introducing Guarded Page Tables (GPTs) and Mountable Merkle Trees (MMTs), enabling the dynamic management of secure memory at scale. It supports thousands of concurrent enclaves and very large memory footprints (up to 512 GB), making it particularly suited to serverless computing workloads. This stands in

contrast to earlier systems like SGX, where enclave memory is capped (e.g., 256 MB in the EPC) and scalability for containerized or microservice environments is limited.

Elasticlave attempts to improve scalability in a different dimension by supporting temporal enclaves and memory sharing between enclaves, but its reliance on PMP limits the number of concurrently protected memory regions.

It is important to note that the performance figures cited, such as Penglai's reported 5% overhead for memory-intensive applications, are derived from each proposal's independent evaluation under different experimental setups, benchmarks, and hardware platforms. This limits direct quantitative comparability. However, the architectural trade-offs are clear: lightweight frameworks like Keystone (reporting 1× runtime overhead) and TIMBER-V (25% overhead) are optimized for minimal single-enclave overhead and memory footprint in embedded contexts. In contrast, Penglai targets a different design point: maintaining low per-enclave overhead while scaling to thousands of concurrent enclaves, a capability beyond lighter-weight frameworks. This distinction highlights how different architectures optimize for different deployment scenarios, from resource-constrained embedded systems to scalable cloud platforms. Taken together, only Penglai approaches the requirements of cloud-native workloads, while most others remain suitable for embedded or single-application use cases.

4.3. Secure I/O

Mainstream TEEs such as SGX lack support for secure I/O, forcing enclaves to delegate device access to the untrusted host OS. This delegation introduces significant attack vectors. Several RISC-V proposals attempt to address this gap. CURE introduces enclave-to-peripheral binding through its bus arbiter and filter engine, ensuring that devices are assigned exclusively to enclaves. HECTOR-V further enhances secure I/O by validating transactions based on core and device identifiers enforced by the Security Monitor. While these mechanisms strengthen protection, they do not yet address more complex scenarios such as dynamic binding and unbinding of stateful peripherals, which remains an open research problem.

WorldGuard extends the notion of domains to encompass both agents and resources, allowing for bus-level filtering of device access. However, its focus on static world assignment limits flexibility. Dorami, Keystone, and Sanctum, by contrast, provide no secure I/O binding, leaving enclaves vulnerable to DMA and Iago-style attacks from malicious peripherals. As IoT and edge systems rely heavily on secure sensor integration, this remains one of the most pressing gaps in the RISC-V enclave landscape.

4.4. Defenses Against Side-Channel and Microarchitectural Attacks

Side-channel attacks remain one of the most persistent threats against enclaves, with defenses evolving from basic cache management to speculative execution protection. Sanctum pioneered cache partitioning and flush-on-context-switch mechanisms [35], while CURE extended this with cache tagging and L2 way partitioning [45]. TIMBER-V introduced memory tagging, but lacks systematic cache leakage defenses [40]. However, these cache-centric approaches address only specific leakage channels without tackling the root cause of speculative execution vulnerabilities.

The challenge is fundamentally architectural: comprehensive protection requires either eliminating speculation, with prohibitive performance costs, or implementing complete microarchitectural state cleansing across all components (caches, TLBs, branch predictors, issue queues, and load-store buffers). Current proposals fall short of this ideal. MI6's purge instructions for microarchitectural buffers represent progress, but the implementation fails to cleanse critical states, such as issue queues, and does not ensure write-back of

dirty cache lines [42]. Furthermore, MI6 remains tied to the RiscyOO processor and lacks shared-memory support.

Beyond performance costs, these countermeasures face significant challenges in verifiability and resilience against adaptive adversaries. Techniques like way-based cache partitioning (CURE) or page coloring (Sanctum) involve complex hardware controllers or software management that are difficult to formally verify for correctness, potentially creating false security guarantees. Furthermore, flushing-based approaches (Keystone, Sanctum) may leave residual states in microarchitectural buffers to not be covered by flush instructions. Adaptive adversaries can exploit these gaps using techniques like Prime+Probe on non-flushed structures or leverage the performance degradation from frequent flushing to mount denial-of-service attacks. Most current countermeasures thus represent best-effort protections rather than proven resilient defenses against determined adversaries with microarchitectural knowledge.

Evidence from real RISC-V cores demonstrates these vulnerabilities are practical, not theoretical. Successful Spectre-v1 and Branch Shadowing attacks have been demonstrated on the BOOM out-of-order processor [63], proving that speculative execution can bypass TEE isolation on actual hardware. This is particularly significant as many academic TEE proposals assume or build upon high-performance cores like BOOM.

Other approaches exhibit similar limitations. SPEAR-V focuses narrowly on same-core leakage while ignoring cross-core channels [54], and Keystone relies only on L1 cache flushing, leaving shared last-level caches unprotected [30]. Critically, no RISC-V TEE integrates a comprehensive suite of low-overhead countermeasures, such as speculative load hardening, secure speculation designs, and complete state cleansing, into a unified defense.

Beyond traditional side-channel attacks, RISC-V TEEs face emerging threats that demand hardware–software co-designed countermeasures. Fault injection attacks represent a critical threat vector where adversaries manipulate environmental conditions, such as voltage, clock frequency, or temperature, to induce computational errors and bypass security checks. These attacks can undermine cryptographic operations, authentication mechanisms, and control flow integrity. Effective countermeasures require both circuit-level hardening through dual modular redundancy and protocol-level protections incorporating temporal redundancy and integrity verification. The CLKSCREW attack demonstrated against ARM TrustZone [17] illustrates the feasibility of such attacks, highlighting the need for robust power management and clock integrity verification in RISC-V TEE implementations.

Physical memory attacks, including bus snooping and cold-boot attacks, target data at rest in main memory. While some RISC-V TEEs like CoVE incorporate memory encryption, most academic proposals lack comprehensive memory encryption with integrity protection; a capability essential for defending against physical adversaries. The absence of hardware-enforced memory encryption engines with integrity trees, as found in commercial solutions like AMD SEV-SNP and Intel TDX, leaves RISC-V TEEs vulnerable to direct memory extraction and modification attacks.

DMA attacks from malicious peripherals constitute another significant threat class, where compromised devices with direct memory access capabilities can read or modify enclave memory. Frameworks like CURE and HECTOR-V have begun addressing this through bus-level filtering and access control mechanisms, but these solutions remain preliminary and lack comprehensive verification. The scientific challenge involves designing formally verifiable IOMMU-like structures that can enforce fine-grained access policies without introducing prohibitive latency overheads.

Logical and protocol-level attacks, particularly Iago attacks where an untrusted OS manipulates system call returns to subvert enclave execution, persist in systems that rely on external services for privileged operations. Keystone has demonstrated susceptibil-

ity to such attacks, and similar vulnerabilities likely exist in other OS-dependent TEE architectures. Comprehensive mitigation strategies must either minimize external dependencies through self-contained TEE designs or implement rigorous input validation, state continuity checks, and attestation of system call results.

Denial-of-service attacks, while often considered outside traditional TEE security guarantees, can undermine system availability in critical applications. These attacks may target shared resources, exhaust enclave memory allocations, or exploit performance degradation from security mechanisms. Countermeasures require resource partitioning, quality-of-service enforcement at the hardware level, and admission control policies that prevent resource exhaustion.

The absence of integrated solutions addressing this multi-faceted threat landscape, combined with demonstrated vulnerabilities in foundational RISC-V cores, means no current system provides comprehensive protection. Future RISC-V TEE architectures must adopt a holistic security approach that addresses the full spectrum of cache, speculative, physical, and logical attacks through verifiable hardware–software co-design, establishing defense-in-depth against increasingly sophisticated adversaries.

4.5. Systemic Challenges

Several systemic challenges are evident across the surveyed works. First, fragmentation is a critical issue: the openness of RISC-V allows for highly customized enclave architectures, but this diversity results in incompatibility across implementations and prevents the emergence of a unified programming model. Second, the risk of Iago attacks persists wherever enclaves rely on the untrusted host OS to service system calls [23]. Keystone has been shown to be susceptible to this class of attacks, and similar risks exist in other proposals that adopt OS-assisted system interfaces. Third, while some designs claim to reduce the Trusted Computing Base (TCB), the assumption that components such as the Security Monitor or enclave runtimes are bug-free is unrealistic. Bugs in these privileged components would undermine the guarantees of isolation and attestation. Finally, most proposals remain at the prototype stage, often evaluated only in simulation or on FPGA platforms, with limited validation on production silicon or under adversarial workloads.

Our comparison also highlights gaps in compliance and certification. None of the surveyed proposals integrates GlobalPlatform APIs or PKCS#11, and no certification efforts exist for RISC-V enclaves. Furthermore, cryptographic support is underdeveloped: most frameworks rely on software-only crypto, with only HECTOR-V partially exploring hardware acceleration. This limits applicability in domains like financial services or 5G, where certified crypto and compliance are mandatory.

4.6. Toward a Cohesive Ecosystem

Taken together, these findings underscore that the RISC-V enclave and TEE landscape is still in a formative stage. The openness and extensibility of the ISA enable diverse architectural explorations, but the lack of convergence hinders broader adoption. Future work must balance three dimensions simultaneously: (i) rigorous security guarantees against advanced attacks, (ii) efficient performance and scalability for practical use, and (iii) usable software ecosystems with strong standards compliance.

Establishing reference implementations, fostering upstream support in projects such as OpenSBI, Linux, and OP-TEE, and ensuring interoperability between enclave models are key milestones. Ultimately, bridging the gap between academic prototypes and industrial deployments will determine whether RISC-V can evolve into a cohesive and trustworthy foundation for confidential computing.

4.7. Toward a Unified Programming Model for RISC-V TEEs

A major limitation across current RISC-V TEE proposals is the absence of a unified programming and execution model. Each framework, such as Keystone, CURE, or CoVE, defines its own enclave lifecycle, ABI, and SDK, which hinders portability and fragments the developer ecosystem. For instance, Keystone relies on a custom enclave runtime (Eyrie) and SBI-based monitor interface, while CoVE and AP-TEE adopt virtualization-based ABIs tailored for confidential virtual machines. This diversity forces developers to rewrite trusted applications for each TEE implementation, increasing development costs and limiting adoption.

To address this fragmentation, we propose a layered software architecture that decouples the developer-facing API from the underlying TEE implementation, while aligning with the RISC-V privilege model and industry standards. This model is structured as follows:

At the highest level, application developers write TAs using a high-level SDK that exposes a standardized API, ideally a RISC-V-specific profile of the GlobalPlatform TEE Client API. This allows for developers to code against a portable, well-defined interface for secure operations such as sealed storage, cryptography, and attestation, without requiring knowledge of the underlying TEE hardware.

The compiled TA executes within a Portable TEE Runtime, an enhanced version of runtimes such as Keystone's Eyrie or a new standards-compliant runtime. This runtime operates inside the enclave at the Supervisor (S) or User (U) privilege level and is responsible for implementing the GlobalPlatform TEE Internal API. It manages the TA's lifecycle, memory, and system calls within the secure context, while remaining agnostic to the underlying security monitor.

The runtime communicates with the hardware via a thin, standardized TEE Management Abstraction Layer, implemented as a set of RISC-V SBI Extensions. These extensions provide a unified hypercall interface for critical operations such as enclave creation, context switching, attestation, and resource management. This layer abstracts whether the underlying Security Monitor uses PMP, MTT, or other isolation mechanisms, enabling interoperability across TEE implementations.

At the lowest software level, the TEE-Specific SM resides in Machine (M) mode and is implementation-specific (e.g., Keystone's, CURE's, or Penglai's monitor). Under this model, the SM need only comply with the standardized SBI extensions, ensuring that it can serve multiple portable runtimes without modification.

This layered architecture formally decouples the developer API from the hardware implementation, enabling a single TA binary, compiled for the base RISC-V ISA and linked against the standard runtime, to execute on any compliant TEE platform. By aligning with emerging RISC-V SBI standards such as TSM and RPMI, this model also supports attestation, key management, and inter-enclave communication in a verifiable and architecture-independent manner. Adopting this approach would directly address the current fragmentation, foster a unified developer ecosystem, and accelerate the adoption of RISC-V TEEs in production environments.

4.8. Compliance and Standardization

Compliance with industry specifications remains largely unaddressed in the current RISC-V TEE landscape. None of the surveyed proposals demonstrates full conformance with established standards such as GlobalPlatform APIs or PKCS#11, which are widely required in regulated domains such as automotive, financial, and mobile security, despite some early efforts that remain largely theoretical or in an initial phase [64,65]. This absence presents a significant barrier to adoption, as developers must rewrite trusted applications

for each specific RISC-V TEE implementation, preventing portability from established platforms like ARM TrustZone and Intel SGX.

A critical interoperability challenge lies in the integration of standardized cryptographic APIs, particularly PKCS#11. The architectural placement of the PKCS#11 module within the TEE software stack presents a key research problem. Our analysis identifies the following two primary, non-exclusive research paths for this integration:

In the first approach, PKCS#11 is implemented as a Trusted Service. The PKCS#11 library itself would be realized as a privileged TA. This service TA would manage cryptographic keys within the TEE's secure storage and perform operations using the TEE's internal cryptographic APIs. Other TAs would then communicate with this service via secure inter-enclave communication mechanisms—a capability that remains an active research area, as explored by systems like Elasticlave.

The second approach integrates PKCS#11 directly within the Portable TEE Runtime. Here, the PKCS#11 standard would be implemented as part of the enclave runtime (the enclave OS), providing a more tightly integrated and potentially higher-performance solution. This model allows for cryptographic operations to be directly mapped to available hardware accelerators, but it also increases the complexity and size of the runtime's TCB.

The scientific challenge underlying these models involves formally verifying their security properties, ensuring that the PKCS#11 interface does not introduce new attack surfaces, and optimizing the performance overhead associated with marshaling and context-switching between the REE client and the TEE-based PKCS#11 service.

Concrete steps can bridge this compliance gap while preserving RISC-V's architectural flexibility. First, layered abstraction approaches could implement standardized APIs through shim layers or library operating systems that map GlobalPlatform specifications to native RISC-V TEE primitives. Second, software-defined TEE frameworks demonstrate that multiple TEE models can be emulated atop common hardware abstractions, providing a pathway to export uniform APIs. Finally, the RISC-V community could define minimal "TEE Profiles" specifying mandatory features required for standards compliance, balancing interoperability with implementation flexibility.

CoVE and AP-TEE move closer to aligning with confidential computing standards by adopting virtualization-based abstractions, but they still lack standardized interfaces for trusted applications. Defining a standardized, efficient path for integrating industry-standard APIs like PKCS#11 is a prerequisite for the RISC-V TEE ecosystem to mature beyond research prototypes and achieve broad adoption in production environments.

4.9. Trusted Computing Base (TCB) Size

The TCB size varies dramatically across proposals. Sanctum and Keystone maintain relatively small monitors (5–8 k LoC), which facilitates formal verification but, limits feature richness. Penglai and Elasticlave expand the TCB to over 10k LoC to support scalability and memory sharing, increasing the verification burden. CURE and HECTOR-V integrate hardware components into their TCB, while systems like MI6 and Cerberus provide only partial or unquantified measurements. Although a smaller TCB is desirable for verification, a too minimal TCB may offload responsibilities to untrusted OS components, reintroducing Iago attack surfaces. Balancing minimalism with functionality remains an unresolved challenge.

4.10. Hardware Modifications and Deployability

The surveyed RISC-V TEE architectures reveal a fundamental tension between security guarantees and real-world deployability, directly reflected in their hardware modification

requirements. Our analysis categorizes these requirements into three distinct levels, each with implications for portability, verification, and adoption.

Systems requiring no hardware modifications, such as Keystone, DORAMI, and SPEAR-V, rely exclusively on standard RISC-V features like Physical Memory Protection (PMP) and existing privilege levels. These frameworks prioritize deployability and can run on commercial RISC-V cores without vendor cooperation. However, this approach often limits security guarantees to software-enforced isolation, making them vulnerable to microarchitectural attacks and dependent on the correct implementation of complex software monitors.

Architectures with minor hardware modifications, including Sanctum, CoVE, and Penglai, introduce targeted changes to specific components such as page table walkers, cache controllers, or memory management units. These modifications enhance security without fundamentally altering the processor architecture, maintaining reasonable portability while providing stronger isolation guarantees. For instance, Sanctum's modified page table walker prevents OS access to enclave memory, while CoVE's Memory Tracking Table enables fine-grained page-level isolation.

Frameworks demanding structural hardware modifications, such as HECTOR-V, TIMBER-V, CURE, and WorldGuard, require significant architectural changes including new processor cores, tagged memory systems, custom bus filters, or heterogeneous core architectures. These systems offer the strongest security properties, often including secure I/O paths and comprehensive side-channel protection, but at the cost of limited portability and high implementation barriers.

The implementation platform further influences real-world relevance. Most academic proposals are validated on FPGA platforms, providing proof-of-concept but limited performance characterization. Simulator-based evaluations, as seen in TIMBER-V and SPEAR-V, offer flexibility for architectural exploration but lack real hardware validation. The absence of ASIC implementations across surveyed works highlights the maturity gap between research prototypes and production-ready solutions.

This graded analysis reveals that no single approach optimally balances security, performance, and deployability. The choice between these design points represents a fundamental trade-off that depends on target deployment scenarios, threat models, and available hardware resources.

4.11. Cryptographic Support

Cryptographic acceleration remains critically underdeveloped across current RISC-V TEE proposals, representing a significant barrier to practical deployment in performance-sensitive and high-assurance environments. While most frameworks, including Keystone, Sanctum, and Penglai, rely entirely on software-based cryptographic implementations, this approach raises substantial concerns for high-throughput workloads such as TLS termination, blockchain operations, and encrypted databases. The absence of dedicated cryptographic hardware not only impacts performance, but also increases vulnerability to timing side-channels in software implementations. Our analysis identifies three distinct, interconnected research layers that must be addressed to achieve robust cryptographic support in future RISC-V TEEs.

The most immediate opportunity lies in the architectural integration of RISC-V's standardized cryptographic extensions. The Scalar Cryptography (Zk) extension suite, comprising sub-extensions such as Zkne (AES), Zknd (SM4), and Zknh (SHA-2), provides fundamental building blocks for efficient cryptographic operations. Similarly, the Vector Cryptography extensions (Zvkn, Zvkg, etc.) enable parallel processing of cryptographic workloads. The research challenge extends beyond mere ISA support to encompass the secure management of cryptographic registers and state during enclave context switches.

Future TEE architectures must ensure that these microarchitectural states are properly isolated, flushed, or partitioned to prevent leakage across security domains. Furthermore, research is needed to develop secure methods for exposing these accelerated instructions to TAs, whether through compiler intrinsics, managed runtime APIs, or monitor-mediated services, without unduly expanding the TCB.

For high-throughput and latency-sensitive applications, dedicated memory-mapped cryptographic coprocessors present a compelling research direction. These specialized hardware units would function as secure peripherals, accessible exclusively to the SM or authorized enclaves through a secure bus fabric—a mechanism preliminarily explored by systems like HECTOR-V. The key research question involves designing a verifiable driver model for these coprocessors that operates within the TEE's trust boundary. Such a model must prevent Iago-style attacks on control registers and ensure that cryptographic keys never leave the protected environment. This approach would enable efficient offloading of computationally intensive operations like public-key cryptography and bulk encryption while maintaining strong isolation guarantees.

The most significant cryptographic gap in current RISC-V TEEs is the absence of transparent, hardware-enforced memory encryption with integrity protection—a capability central to commercial counterparts like AMD SEV-SNP and Intel TDX. We identify the co-design of a Memory Encryption Engine (MEE) with integrity trees as a paramount research challenge for the RISC-V ecosystem. A viable solution requires the integration of an on-chip key manager, hardware acceleration for encryption algorithms (such as AES-XTS), and the use of RISC-V PMAs to designate memory regions for cryptographic protection. Such a system would provide a fundamental layer of confidentiality and integrity against physical attackers and powerful system-level adversaries, addressing a critical limitation in current academic proposals.

Only CoVE among the surveyed frameworks explicitly introduces replay-protected memory encryption, approaching the guarantees of commercial TEEs. Bridging this cryptographic gap will require coordinated advances across multiple layers of the system stack, from ISA extensions to SoC-level security architectures. The structured research agenda outlined here, spanning standardized ISA extensions, dedicated coprocessors, and transparent memory encryption, provides a clear pathway toward achieving cryptographic capabilities that are both performant and verifiably secure within the RISC-V trusted execution landscape.

4.12. Grouping by Enclave Type and Memory Isolation

The table also reveals important differences in enclave types and isolation models. User-level enclaves (Sanctum, Penglai) simplify programmability, but depend on the OS for privileged operations, creating Iago vulnerabilities. Multi-level designs (CURE, Keystone, TIMBER-V) introduce supervisor or machine-level enclaves, expanding functionality, but enlarging the TCB. Core-partitioned approaches (HECTOR-V, WorldGuard) eliminate sharing, but suffer from rigid resource duplication. Memory isolation is equally diverse: PMP and ePMP dominate lightweight designs (Keystone, Dorami, Elasticlave, SPEAR-V), while Penglai relies on MMU-based page isolation, CURE employs cache tagging, TIMBER-V introduces memory tagging, and CoVE adopts a virtualization-based Memory Tracking Table (MTT). These variations reflect a lack of consensus on the "right" abstraction for isolation, with each design optimizing for a different axis: programmability, performance, or security.

4.13. Cross-Cutting Observations

Synthesizing across these dimensions, several patterns emerge. Proposals that minimize hardware changes (Keystone, Dorami, Elasticlave) are easier to deploy, but weaker against advanced attacks. Designs that introduce strong defenses (CURE, HECTOR-V, CoVE) achieve richer isolation at the cost of portability. Scalability remains the do-

main of Penglai, but its complexity and large TCB raise practical barriers. Across the board, SDK maturity, compliance, and cryptographic integration remain glaring omissions, underscoring that RISC-V TEEs are still at a research–prototype stage rather than production-ready solutions.

4.14. Architectural Components for a Verifiable RISC-V TEE

The survey of existing RISC-V TEE proposals reveals that achieving a system that is simultaneously secure, efficient, and verifiable requires a coherent architectural synthesis of the most promising approaches while addressing their identified limitations. We propose a blueprint comprising four foundational pillars that must be co-designed to meet these objectives.

The cornerstone of this architecture is a formally verified, minimal SM operating in Machine mode. This component must be designed with verification as a primary constraint, implementing only the essential functions of enclave lifecycle management, context switching, memory isolation enforcement, and remote attestation. Using formal verification tools such as Coq or Isabelle, the core state transitions and security invariants of the monitor can be mathematically proven to maintain isolation and integrity properties. This directly addresses the verification challenges observed in systems like Keystone and Sanctum, where complex monitors resist comprehensive analysis and introduce unquantified trust assumptions.

The second component consists of hardware-enforced, multi-granular isolation primitives that operate synergistically across different system layers. At the memory level, enhanced PMP or a dedicated MPU must provide efficient, fine-grained memory partitioning for lightweight enclaves, overcoming the scalability limitations observed in PMP-based systems like Keystone. For microarchitectural isolation, the architecture must integrate spatial partitioning mechanisms such as way-based cache allocation alongside temporal isolation through precise state cleansing instructions. This combined approach addresses both cache-based and speculative execution side-channels while maintaining performance efficiency through selective application of these costly operations.

The third critical element is integrated cryptographic acceleration and memory encryption. To achieve both security and performance, the architecture must deeply integrate RISC-V's scalar and vector cryptography extensions into the enclave execution context, ensuring that their registers and states are properly managed during context switches. Furthermore, a transparent Memory Encryption Engine with integrity protection is essential to safeguard off-chip memory from physical attacks and DMA-based exploits—a capability notably absent in most academic proposals, but critical for matching commercial TEE security guarantees.

The final component addresses ecosystem fragmentation through a standardized, layered software stack that separates security concerns while enabling verification. A standardized TEE Management Application Binary Interface, implemented as RISC-V SBI extensions, provides a uniform interface for enclave operations across different hardware implementations. This abstraction enables the development of portable, potentially verified enclave runtimes that can host TAs written against standard APIs such as GlobalPlatform's TEE Internal API. This layered approach decouples application security from hardware specifics, enabling independent verification of each component while maintaining interoperability across different RISC-V TEE implementations.

This architectural blueprint demonstrates that a secure, efficient, and verifiable RISC-V TEE is fundamentally a systems problem requiring coordinated advances across hardware, firmware, and software layers. By synthesizing the strongest elements of existing proposals while systematically addressing their limitations, this framework provides a concrete foundation for future research and development in the RISC-V trusted computing ecosystem.

4.15. Deployment Scenarios and Lightweight TEEs

The diverse landscape of RISC-V TEE proposals reflects fundamentally different security assumptions, design priorities, and resource constraints across deployment environments. Understanding these contextual factors is essential for evaluating the practical relevance and applicability of each architecture. We analyze three primary deployment scenarios—IoT/embedded systems, edge computing, and cloud/server environments—each with distinct requirements that shape TEE design choices.

IoT and embedded systems operate under extreme constraints, including severe power and area limitations, real-time execution requirements, and minimal memory footprints. In these environments, lightweight TEEs such as TIMBER-V and DORAMI prioritize minimal trusted computing bases and low performance overhead, often sacrificing advanced security features for deployability. These systems typically focus on core memory isolation using standard PMP mechanisms while forgoing comprehensive side-channel protection and secure I/O capabilities. The design philosophy emphasizes simplicity and verification, with TCB sizes often under 10 k lines of code and performance overhead targets below 10%. These trade-offs are acceptable, given the constrained threat models and resource limitations of IoT devices.

Edge computing environments present a middle ground with moderate computational resources, but critical requirements for secure peripheral interactions. Systems like CURE and HECTOR-V address these needs by introducing secure I/O paths and peripheral binding mechanisms while maintaining reasonable performance profiles. These designs balance the need for sensor data integrity, secure communication with external devices, and protection against DMA attacks. The TCB expands to include hardware filtering logic and access control mechanisms, with performance overhead typically ranging from 15–25%. This represents a pragmatic compromise between security assurance and practical deployability in resource-constrained but security-sensitive edge applications.

Cloud and server environments demand high performance, scalability to thousands of concurrent enclaves, and strong isolation against sophisticated adversaries. Architectures such as Penglai, CoVE, and AP-TEE focus on scalable memory management, support for confidential virtual machines, and comprehensive defenses against microarchitectural attacks. These systems tolerate larger TCBs and higher complexity in exchange for features like memory encryption, dynamic enclave creation, and attestation chains spanning multiple trust domains. Performance overhead can reach 5–15% for memory-intensive workloads, but this is acceptable given the security benefits and resource availability in cloud settings.

The progression from IoT to cloud environments reveals a clear trade-off between security feature richness and resource constraints. Lightweight TEEs for embedded systems prioritize minimalism and verification, while cloud-oriented architectures embrace complexity for stronger isolation and scalability. This analysis demonstrates that no single TEE design optimally serves all deployment scenarios, highlighting the importance of context-aware security architecture and the continued need for specialized solutions across the computing spectrum.

4.16. Implementation Readiness and Production Viability

The surveyed RISC-V TEE proposals span a wide spectrum of implementation maturity, from academic prototypes to near-production solutions. To systematically assess practical deployability, we introduce a unified evaluation framework that synthesizes multiple dimensions of implementation readiness. This framework integrates software ecosystem maturity, compliance and certification status, hardware implementation complexity, and evidence of real-world validation.

Software ecosystem maturity varies significantly across proposals. Systems like Keystone and CoVE offer partial SDK support and development toolchains, enabling initial application development and research prototyping. However, no surveyed system provides comprehensive integration with industry-standard APIs such as GlobalPlatform TEE Client API or PKCS#11, limiting the portability of existing trusted applications. The absence of mature software ecosystems represents a critical barrier for production deployment, as developers require stable interfaces, debugging tools, and documentation to build and maintain secure applications.

Compliance and certification aspects remain largely unaddressed across the RISC-V TEE landscape. None of the surveyed proposals demonstrate conformance with established standards required in regulated industries such as automotive (ISO 26262), [66] financial services (FIPS 140-3 [67]), or mobile security (GlobalPlatform TEE certification). This gap highlights the immaturity of RISC-V TEEs for safety-critical and high-assurance applications where certified implementations are mandatory. Early efforts toward standardization, such as the AP-TEE specification, represent important first steps, but have not yet achieved formal certification.

Hardware implementation complexity directly impacts production viability. Systems requiring no hardware modifications (Keystone, DORAMI, SPEAR-V) offer the highest deployability, running on commercial RISC-V cores without vendor cooperation. Architectures with minor modifications (Sanctum, CoVE, Penglai) maintain reasonable portability while enhancing security guarantees. However, systems demanding structural changes (HECTOR-V, TIMBER-V, CURE) face significant adoption barriers due to the need for custom silicon or extensive FPGA modifications. The predominance of FPGA-based validation across all proposals further indicates the research-stage nature of current implementations, with no surveyed system demonstrating ASIC implementation or large-scale deployment.

Evidence of industrial adoption remains limited, with most proposals originating from academic institutions and lacking documented production deployments. The transition from research prototypes to commercially viable solutions not only requires technical maturity, but also ecosystem support, long-term maintenance commitments, and demonstrable interoperability with existing infrastructure. The emerging collaboration between academic researchers and industry consortia, particularly within RISC-V International working groups, suggests growing recognition of these practical requirements.

This unified framework reveals that while significant architectural innovation has occurred in RISC-V TEE research, substantial work remains to bridge the gap between academic prototypes and production-ready solutions. Future efforts must address the complete stack of requirements, from hardware implementation and software ecosystems to compliance certification and industry adoption, to realize the promise of open, verifiable trusted execution in real-world deployments.

4.17. Future Directions for RISC-V TEEs and Secure Enclaves

Research and development on RISC-V TEEs remains highly active, with both academic initiatives and contributions from the RISC-V community and hardware vendors seeking to reduce fragmentation and move toward standardized solutions. Several directions are likely to guide future progress. One critical focus will be the development of hardware-assisted defenses against side-channel attacks: since most leakage arises from shared microarchitectural components, software countermeasures alone are insufficient, and RISC-V's extensibility provides an opportunity to integrate primitives such as cache partitioning, memory tagging, and speculation control directly into hardware. Another promising direction is the use of RISC-V cores as coprocessors within heterogeneous SoCs, where they can be dedicated to trusted execution tasks; early industrial efforts already demonstrate the feasibility of this approach. Finally, the growing deployment of RISC-V in embedded and

edge computing highlights the need for lightweight TEEs tailored to resource-constrained devices, where traditional designs are impractical. Projects such as TIMBER-V illustrate this trajectory, pointing toward enclave models that emphasize scalability, low overhead, and adaptability to pervasive IoT contexts.

Beyond these architectural advancements, RISC-V TEEs have significant potential to enable transformative capabilities across several interdisciplinary domains. In AI security and privacy-preserving machine learning, frameworks like CoVE and AP-TEE can protect model integrity during inference by ensuring that proprietary algorithms and parameters remain confidential even on untrusted cloud infrastructure. During training, enclaves, such as those provided by Keystone, can secure sensitive datasets and enable confidential collaborative learning across multiple institutions while preventing model extraction or data leakage. The integration of TEEs with homomorphic encryption and secure multi-party computation could further enhance privacy guarantees for distributed AI workflows.

In federated learning scenarios, RISC-V TEEs like Penglai can provide trusted environments for secure model aggregation by verifying the integrity of participant updates and preventing poisoning attacks from malicious clients. Enclaves can enforce differential privacy mechanisms while maintaining audit trails for regulatory compliance. This approach is particularly valuable in healthcare applications where patient data must remain decentralized, but model training requires trustworthy aggregation across multiple hospitals or research institutions.

Additional promising application domains include blockchain and decentralized finance, where systems like CURE and WorldGuard can secure smart contract execution and protect private key management from compromised host environments. In confidential data analytics, enclaves enable secure processing of sensitive information across finance, healthcare, and government sectors while maintaining data sovereignty and regulatory compliance. For industrial IoT and automotive systems, TEEs such as HECTOR-V provide hardware-rooted trust for secure over-the-air updates, sensor data integrity, and safety-critical function isolation. Lightweight enclaves like DORAMI and Keystone in secure IoT gateways can isolate telemetry aggregation and credential storage in edge devices, ensuring that compromised firmware cannot access sensitive sensor data or cryptographic keys.

Collectively, these interdisciplinary applications demonstrate that the next generation of RISC-V TEEs will need to balance resilience against advanced attacks with efficiency and deployability across both cloud-scale and edge-scale environments, while addressing domain-specific security and performance requirements. The advancement of verifiable TEE architectures will be crucial for realizing these applications while maintaining the open and customizable nature of the RISC-V ecosystem.

5. Conclusions

This survey presents the first comprehensive analysis of RISC-V trusted execution environments and secure enclaves, systematically mapping their architectural principles, security mechanisms, and implementation maturity. Through a structured taxonomy that categorizes security approaches across microarchitectural, hardware, and system-software levels, we have illuminated the fundamental trade-offs between security assurance, performance overhead, and practical deployability. Our analysis demonstrates that while significant innovation has occurred in RISC-V confidential computing, the ecosystem remains fragmented, with substantial gaps between academic prototypes and production-ready solutions.

The comparative framework introduced in this work reveals several critical insights. First, the graded spectrum of hardware modification requirements, from none to structural changes, directly impacts real-world adoption potential, with software-centric approaches offering immediate deployability at the cost of comprehensive security guarantees. Second, the absence of standardized programming models and compliance certification represents

a major barrier to ecosystem cohesion and application portability. Third, emerging interdisciplinary applications in AI security, federated learning, and confidential computing demand specialized TEE architectures tailored to specific deployment scenarios, ranging from resource-constrained IoT devices to scalable cloud platforms.

Beyond synthesizing the current state of research, this survey provides a foundation for future advancements in open-hardware confidential computing. The unified evaluation framework for implementation readiness enables researchers and practitioners to systematically assess production viability across multiple dimensions, including software ecosystem maturity, compliance certification, and hardware implementation complexity. The identified research directions, from hardware-assisted side-channel defenses to standardized APIs and verifiable security monitors, chart a course toward cohesive, trustworthy confidential computing on RISC-V.

As the RISC-V ecosystem continues to mature, this survey serves as both a reference for understanding existing TEE architectures and a roadmap for developing next-generation solutions that balance security, efficiency, and deployability across diverse computing environments. The unique opportunities presented by RISC-V's openness and extensibility position it as a transformative platform for building verifiable, transparent trusted computing foundations for the future.

Author Contributions: Conceptualization, M.B.; investigation, M.B.; analysis, M.B.; benchmarking, M.B.; drafting the manuscript, M.B.; writing—review and editing, M.B.; supervision, B.Z. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding. The APC was funded by the authors.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Acknowledgments: The authors gratefully acknowledge support from NXP, which provided the required hardware resources (e.g., FPGAs and RISC-V boards) used in this study.

Conflicts of Interest: The authors declare no conflicts of interest. The funders had no role in the selection, analysis, or interpretation of the literature; in the preparation of the manuscript; or in the decision to publish the results.

References

1. Sabt, M.; Achemlal, M.; Bouabdallah, A. Trusted Execution Environment: What It Is, and What It Is Not. In Proceedings of the IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), Helsinki, Finland, 20–22 August 2015; IEEE: Piscataway, NJ, USA, 2015; pp. 57–64. [CrossRef]
2. Costan, V.; Devadas, S. Intel SGX Explained. In Proceedings of the USENIX Security Symposium, Austin, TX, USA, 10–12 August 2016; USENIX Association: Berkeley, CA, USA, 2016; pp. 175–190.
3. Kaplan, D.; Powell, J.; Woller, T. AMD Memory Encryption. AMD White Paper. 2016. Available online: https://www.amd.com/system/files/TechDocs/55766_SEV-KM_API_Spec.pdf (accessed on 30 September 2025).
4. ARM Ltd. ARM Security Technology—Building a Secure System Using TrustZone Technology. ARM Technical Report. 2009. Available online: https://developer.arm.com/documentation/PRD29-GENC-009492C?lang=en (accessed on 30 September 2025).
5. Intel Corporation. Intel Software Guard Extensions (Intel SGX) Programming Reference. Intel Developer Manual. 2020. Available online: https://www.intel.com/sgx (accessed on 27 September 2025).
6. Oliner, A.; Weisse, O.; Austin, T. Analyzing the Limits of Intel SGX for Secure Cloud Computation. In Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), Madison, WI, USA, 24–26 March 2019; IEEE: Piscataway, NJ, USA, 2019; pp. 233–244. [CrossRef]
7. Kaplan, D.; Powell, J.; Woller, T. *AMD Memory Encryption Technologies*; AMD White Paper; Advanced Micro Devices, Inc.: Santa Clara, CA, USA, 2016.
8. Hetzelt, F.; Buhren, R. SEVered: Subverting AMD's Virtual Machine Encryption. In Proceedings of the 11th European Workshop on Systems Security (EuroSec), Porto, Portugal, 23–26 April 2018; ACM: New York, NY, USA, 2018; pp. 1–6. [CrossRef]
9. Alves, T.; Felton, D. *TrustZone: Integrated Hardware and Software Security*; ARM White Paper; ARM: Cambridge, UK, 2004.

10. Pinto, S.; Santos, N. Demystifying Arm TrustZone: A Comprehensive Survey. *ACM Comput. Surv.* **2019**, *51*, 130. [CrossRef]
11. Azab, A.; Ning, P.; Shah, J.; Bhutkar, R.; Ganju, K.; Shen, W.; Wang, Y. Hypervision Across Worlds: Real-time Kernel Protection from the ARM TrustZone Secure World. In Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS), Scottsdale, AZ, USA, 3–7 November 2014; ACM: New York, NY, USA, 2014; pp. 90–102. [CrossRef]
12. Ge, Q.; Yarom, Y.; Cock, D.; Heiser, G. A Survey of Microarchitectural Timing Attacks and Countermeasures on Modern Processors. *ACM Comput. Surv.* **2019**, *54*, 3–29.
13. Van Bulck, J.; Minkin, M.; Weisse, O.; Genkin, D.; Kasikci, B.; Piessens, F.; Silberstein, M.; Wenisch, T.F.; Yarom, Y.; Strackx, R. Foreshadow: Extracting the Keys to the Intel SGX Kingdom with Transient Out-of-Order Execution. In Proceedings of the 27th USENIX Security Symposium, Baltimore, MD, USA, 15–17 August 2018; USENIX Association: Berkeley, CA, USA, 2018; pp. 991–1008.
14. Van Bulck, J.; Piessens, F.; Strackx, R. LVI: Hijacking Transient Execution through Microarchitectural Load Value Injection. In Proceedings of the IEEE Symposium on Security and Privacy (SP), San Francisco, CA, USA, 18–21 May 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 54–72. [CrossRef]
15. Chen, T.; Zang, B.; Chen, H.; Guan, H. TruSpy: Cache Side-Channel Information Leakage from the Secure World on ARM Devices. *IACR Cryptol. ePrint Arch.* **2017**, *2017*, 1169. Available online: https://eprint.iacr.org/2017/1169 (accessed on 30 September 2025).
16. Xu, L.; Yang, J.; Li, Z.; Zhao, Y.; Zhang, T.; Wang, T. TruSense: Information Leakage from ARM TrustZone via Cache Side Channels. In Proceedings of the IEEE International Conference on Computer Design (ICCD), Abu Dhabi, United Arab Emirates, 17–20 November 2019; IEEE: Piscataway, NJ, USA, 2019; pp. 471–474. [CrossRef]
17. Tang, A.; Sethumadhavan, S.; Stolfo, S. CLKSCREW: Exposing the Perils of Security-Oblivious Energy Management. In Proceedings of the 26th USENIX Security Symposium, Vancouver, BC, Canada, 16–18 August 2017; USENIX Association: Berkeley, CA, USA, 2017; pp. 1057–1074.
18. Chen, G.; Chen, S.; Xiao, Y.; Zhang, Y.; Lin, Z.; Lai, T.H.; Xing, X. SoK: Understanding Security Vulnerabilities of Software-based Trusted Execution Environments. In Proceedings of the IEEE Symposium on Security and Privacy (SP), San Francisco, CA, USA, 20–22 May 2019; IEEE: Piscataway, NJ, USA, 2019; pp. 1003–1020. [CrossRef]
19. Asanović, K.; Patterson, D. *Instruction Sets Should Be Free: The Case for RISC-V*; Technical Report UCB/EECS-2014-146; EECS Department, University of California: Berkeley, CA, USA, 2014. Available online: https://www2.eecs.berkeley.edu/Pubs/TechRpts/2014/EECS-2014-146.html (accessed on 30 September 2025).
20. Lu, T. A Survey on RISC-V Security: Hardware and Architecture. *arXiv* **2021**, arXiv:2107.04175. [CrossRef]
21. Anders, J.; Andreu, P.; Becker, B.; Becker, S.; Cantoro, R.; Deligiannis, N.I.; Elhamawy, N.; Faller, T.; Hernandez, C.; Mentens, N.; et al. A Survey of Recent Developments in Testability, Safety and Security of RISC-V Processors. In Proceedings of the 2023 IEEE European Test Symposium (ETS), Venezia, Italy, 22–26 May 2023; IEEE: Piscataway, NJ, USA, 2023; pp. 1–10. [CrossRef]
22. Li, M.; Yang, Y.; Chen, G.; Yan, M.; Zhang, Y. SoK: Understanding Design Choices and Pitfalls of Trusted Execution Environments. In Proceedings of the 19th ACM Asia Conference on Computer and Communications Security (ASIA CCS), Singapore, 1–5 July 2024. [CrossRef]
23. Zhang, F.; Zhou, L.; Zhang, Y.; Ren, M.; Deng, Y. Trusted Execution Environment: State-of-the-Art and Future Directions. *J. Comput. Res. Dev.* **2024**, *61*, 243–260. [CrossRef]
24. Maene, P.; Götzfried, J.; de Clercq, R.; Müller, T.; Freiling, F.; Verbauwhede, I. Hardware-Based Trusted Computing Architectures for Isolation and Attestation. *IEEE Trans. Comput.* **2018**, *67*, 361–374. [CrossRef]
25. Muñoz, A.; Ríos, R.; Román, R.; López, J. A survey on the (in)security of trusted execution environments. *Comput. Secur.* **2023**, *129*, 103180. [CrossRef]
26. Schneider, M.; Masti, R.J.; Shinde, S.; Čapkun, S.; Perez, R. SoK: Hardware-supported Trusted Execution Environments. *arXiv* **2022**, arXiv:2205.12742. [CrossRef]
27. Geppert, T.; Deml, S.; Sturzenegger, D.; Ebert, N. Trusted Execution Environments: Applications and Organizational Challenges. *Front. Comput. Sci.* **2022**, *4*, 930741. [CrossRef]
28. Feng, X.; Su, Y.; Liu, J.; Xu, C.; Liu, Y.; Xu, H.; Chen, K. A Survey of Confidential Computing. *IET Commun.* **2023**, *17*, 1234–1249. [CrossRef]
29. GlobalPlatform. TEE System Architecture, Version 1.3; Technical Specification GPD_SPE_009. *GlobalPlatform*, 2022. Available online: https://globalplatform.org/wp-content/uploads/2022/05/GPD_SPE_009-GPD_TEE_SystemArchitecture_v1.3_PublicRelease_signed.pdf (accessed on 22 September 2025).
30. Lee, D.; Kohlbrenner, D.; Shinde, S.; Asanović, K.; Song, D. Keystone: An Open Framework for Architecting Trusted Execution Environments. In Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys), Heraklion, Greece, 27–30 April 2020; ACM: New York, NY, USA, 2020; pp. 1–16.
31. Sahita, R.; Ge, Q.; Liang, J.; Love, E.; Costan, V.; Li, S. CoVE: Confidential Computing Architecture for RISC-V. In Proceedings of the 2023 IEEE Symposium on Security and Privacy (S&P), San Francisco, CA, USA, 22–25 May 2023; IEEE: Piscataway, NJ, USA, 2023; pp. 1–17.

32. Dessouky, G.; Sadeghi, A.-R.; Stapf, E. Enclave Computing on RISC-V: A Brighter Future for Security? In Proceedings of the Workshop on Secure RISC-V Architecture Design (SECRISC-V), Boston, MA, USA, 5–7 April 2020; Volume 20.
33. Le, A.-T. Research of RISC-V Out-of-Order Processor Cache-Based Side-Channel Attacks: Systematic Analysis, Security Models and Countermeasures. Ph.D. Dissertation, University of Electro-Communications, Tokyo, Japan, 2023.
34. Donnini, V. Integration of the DICE Specification into the Keystone Framework. Ph.D. Dissertation, Politecnico di Torino, Torino, Italy, 2023.
35. Costan, V.; Lebedev, I.; Devadas, S. Sanctum: Minimal Hardware Extensions for Strong Software Isolation. In Proceedings of the 25th USENIX Security Symposium, Austin, TX, USA, 10–12 August 2016; USENIX Association: Berkeley, CA, USA, 2016; pp. 857–874.
36. Wong, M.M.; Haj-Yahya, J.H.; Chattopadhyay, A. SMARTS: Secure Memory Assurance of RISC-V Trusted SoC. In Proceedings of the 7th International Workshop on Hardware and Architectural Support for Security and Privacy (HASP), Los Angeles, CA, USA, 2 June 2018; ACM: New York, NY, USA, 2018; pp. 1–8. [CrossRef]
37. Krentz, K.F.; Voigt, T. Reducing trust assumptions with OSCORE, RISC-V, and Layer 2 one-time passwords. In *International Symposium on Foundations and Practice of Security*; Springer Nature: Cham, Switzerland, 2022; pp. 389–405.
38. Cheang, K.; Rasmussen, C.; Lee, D.; Kohlbrenner, D.W.; Asanović, K.; Seshia, S.A. Verifying RISC-V Physical Memory Protection. *arXiv* **2022**, arXiv:2211.02179. . [CrossRef]
39. Vilanova, L.; Weisse, O.; Bartolini, A.; Bartolini, M.; Sampaio, L. HECTOR-V: A Heterogeneous Architecture for Trusted Execution in RISC-V. In Proceedings of the 2020 Design, Automation & Test in Europe Conference & Exhibition (DATE), Grenoble, France, 9–13 March 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 1464–1469.
40. Feng, S.; Aublin, P.-L.; Ta-Min, R.; Felber, P.; Le Métayer, D. TIMBER-V: Tag-Isolated Memory Bringing Enclaves to RISC-V. In Proceedings of the 2021 IEEE 27th International Symposium on High Performance Computer Architecture (HPCA), Seoul, Republic of Korea, 27 February–3 March 2021; IEEE: Piscataway, NJ, USA, 2021; pp. 432–445.
41. Feng, E.; Lu, X.; Du, D.; Yang, B.; Jiang, X.; Xia, Y.; Zang, B.; Chen, H. Scalable Memory Protection in the PENGLAI Enclave. In Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21), Online, 14–16 July 2021; USENIX Association: Berkeley, CA, USA, 2021; pp. 275–294, ISBN 978-1-939133-22-9.
42. Mashtizadeh, A.; Wentzlaff, D. MI6: Secure Enclaves in a Speculative Out-of-Order Processor. In Proceedings of the 2019 ACM/IEEE 46th Annual International Symposium on Computer Architecture (ISCA), Phoenix, AZ, USA, 22–26 June 2019; IEEE: Piscataway, NJ, USA, 2019; pp. 42–55.
43. Kocher, P.; Horn, J.; Fogh, A.; Genkin, D.; Gruss, D.; Haas, W.; Hamburg, M.; Lipp, M.; Mangard, S.; Prescher, T.; et al. Spectre Attacks: Exploiting Speculative Execution. *Commun. ACM* **2020**, *63*, 93–101. [CrossRef]
44. Li, T.; Hopkins, B.; Parameswaran, S. SIMF: Single-Instruction Multiple-Flush Mechanism for Processor Temporal Isolation. *arXiv* **2020**, arXiv:2011.10249. [CrossRef]
45. Bahmani, R.; Knauth, T.; Sammler, M.; Weiser, S.; Fetzer, C. CURE: A Security Architecture with CUstomizable Enclaves. In Proceedings of the 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), Portland, OR, USA, 24–27 June 2019; IEEE: Piscataway, NJ, USA, 2020; pp. 1–14.
46. Stapf, E.; Jauernig, P.; Brasser, F.; Sadeghi, A.-R. In Hardware We Trust? From TPM to Enclave Computing on RISC-V. In Proceedings of the 2021 IFIP/IEEE 29th International Conference on Very Large Scale Integration (VLSI-SoC), Singapore, 4–7 October 2021; IEEE: Piscataway, NJ, USA, 2021; pp. 1–6. [CrossRef]
47. Koh, H.; Choi, M.; Lee, J.; Park, J. DORAMI: Lightweight Trusted Execution with Enhanced PMP on RISC-V. In Proceedings of the 2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA), Edinburgh, UK, 2–6 March 2024; IEEE: Piscataway, NJ, USA, 2024; pp. 745–758.
48. Schneider, M.; Dhar, A.C.; Puddu, I.; Kostiainen, K.; Capkun, S. Composite Enclaves: Towards Disaggregated Trusted Execution. *arXiv* **2020**, arXiv:2010.10416. [CrossRef]
49. Kieu-Do-Nguyen, B.; Nguyen, K.D.; Dang, T.K.; Binh, N.T.; Pham-Quoc, C.; Tran, N.T.; Pham, C.K.; Hoang, T.T. A Trusted Execution Environment RISC-V System-on-Chip Compatible with Transport Layer Security 1.3. *Electronics* **2024**, *13*, 2508. [CrossRef]
50. Chen, Y.; Chen, H.; Chen, S.; Han, C.; Ye, W.; Liu, Y.; Zhou, H. DITES: A lightweight and flexible dual-core isolated trusted execution SoC based on RISC-V. *Sensors* **2022**, *22*, 5981. [CrossRef] [PubMed]
51. Nasahl, J.; Hoang, T.; Pinto, S.; Köpf, B. WorldGuard: Enforcing Strong Isolation for Trusted Execution on RISC-V. In Proceedings of the 2024 IEEE European Symposium on Security and Privacy (EuroS&P), Vienna, Austria, 8–12 July 2024; IEEE: Piscataway, NJ, USA, 2024; pp. 112–127.
52. Hoang, T.T.; Duran, C.; Serrano, R.; Sarmiento, M.; Nguyen, K.D.; Tsukamoto, A.; Suzaki, K.; Pham, C.K. Trusted Execution Environment Hardware by Isolated Heterogeneous Architecture for Key Scheduling. *IEEE Access* **2022**, *10*, 46014–46027. [CrossRef]
53. Pinto, S.; Martins, J.; Rodriguez, M.; Cunha, L.; Schmalz, G.; Moslehner, U.; Dieffenbach, K.; Roecker, T. RISC-V Needs Secure 'Wheels': The MCU Initiator-Side Perspective. *arXiv* **2024**, arXiv:2410.09839. [CrossRef]

54. Shah, H.; Nguyen, K.; El Haji, M.; Dashti, M.; Knauth, T.; Bahmani, R. SPEAR-V: Software-isolated Enclaves on RISC-V. In Proceedings of the 2022 IEEE International Conference on Computer Design (ICCD), Olympic Valley, CA, USA, 23–26 October 2022; IEEE: Piscataway, NJ, USA, 2022; pp. 289–296.
55. Zhang, Y.; Gu, R.; Wang, H.; Yang, Y.; Li, J.; Wang, X. VirTEE: A Secure Virtualization-based TEE for RISC-V. In Proceedings of the 2022 IEEE International Conference on Parallel and Distributed Systems (ICPADS), Nanjing, China, 10–12 January 2022; IEEE: Piscataway, NJ, USA, 2022; pp. 564–572.
56. Yu, J.Z.; Shinde, S.; Carlson, T.E.; Saxena, P. Elasticlave: An Efficient Memory Model for Enclaves. In Proceedings of the 31st USENIX Security Symposium (USENIX Security 22), Boston, MA, USA, 10–12 August 2022; USENIX Association: Berkeley, CA, USA, 2022; pp. 4111–4128.
57. Pan, S.; Peng, X.; Man, Z.; Zhao, X.; Zhang, D.; Yang, B.; Du, D.; Lu, H.; Xia, Y.; Li, X. Dep-TEE: Decoupled Memory Protection for Secure and Scalable Inter-enclave Communication on RISC-V. In Proceedings of the 30th Asia and South Pacific Design Automation Conference (ASPDAC '25), Tokyo, Japan, 20–23 January 2025; ACM: New York, NY, USA, 2025; pp. 454–460. [CrossRef]
58. Stapf, E.; Weiser, S.; Bahmani, R.; Knauth, T.; Fetzer, C. Cerberus: A Memory Isolation Framework for Enclaves. In Proceedings of the 2021 IEEE International Symposium on Secure and Private Execution Environments (SEED), Washington, DC, USA, 20–21 September 2021; IEEE: Piscataway, NJ, USA, 2021; pp. 1–13.
59. Sahita, R.; Ge, Q.; Li, S.; Costan, V. AP-TEE: Application Platform Trusted Execution Environment Specification for RISC-V. RISC-V Summit. 2023. Available online: https://github.com/riscv-non-isa/riscv-ap-tee (accessed on 27 September 2025).
60. Shen, R.; Wu, J.; Chen, Z.; Zuo, C.; Guo, Y.; Chen, H. Penglai: Scaling Enclave Applications with Dynamic Memory Management. In Proceedings of the 16th European Conference on Computer Systems (EuroSys), Edinburgh, UK, 26–28 April 2021; ACM: New York, NY, USA, 2021; pp. 275–290.
61. Dessouky, G.; Cheang, K.; Bhatotia, P.; Weiser, S.; Bahmani, R. AnyTEE: A Portable and Open Framework for Enclaves. In Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security (AsiaCCS), Hong Kong, China, 7–11 June 2021; ACM: New York, NY, USA, 2021; pp. 297–310.
62. Shinde, S.; Knauth, T.; Weisse, O.; Asanović, K.; Song, D. Elasticlave: An Efficient Memory Model for Enclaves. In Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI), Online, 4–6 November 2020; USENIX Association: Berkeley, CA, USA, 2020; pp. 523–539.
63. Le, A.-T.; Dao, B.-A.; Suzaki, K.; Pham, C.-K. Experiment on Replication of Side Channel Attack via Cache of RISC-V Berkeley Out-of-Order Machine (BOOM) Implemented on FPGA. *Electronics* **2020**, *9*, 2009. [CrossRef]
64. Boubakri, M.; Chiatante, F.; Zouari, B. Open Portable Trusted Execution Environment Framework for RISC-V. In Proceedings of the 2021 IEEE 19th International Conference on Embedded and Ubiquitous Computing (EUC), Shenyang, China, 20–22 October 2021; pp. 1–8. [CrossRef]
65. Suzaki, K.; Nakajima, K.; Oi, T.; Tsukamoto, A. Library Implementation and Performance Analysis of GlobalPlatform TEE Internal API for Intel SGX and RISC-V Keystone. In Proceedings of the 2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), Guangzhou, China, 29 December 2020–1 January 2021; pp. 200–207. [CrossRef]
66. *ISO 26262*; Road Vehicles—Functional Safety, 2nd ed. ISO: Geneva, Switzerland, 2018; Parts 1–12.
67. *FIPS PUB 140–3*; Security Requirements for Cryptographic Modules. U.S. Department of Commerce: Gaithersburg, MD, USA, 2019.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.